

Republic of Tunisia
Ministry of Higher Education and
Scientific Research

University of Sfax
Faculty of Sciences of Sfax



Computer Science and Communication
Department

Graduation Project Report
Licence in Computer Science

GRADUATION PROJECT REPORT

Submitted to

Faculty of Sciences of Sfax
Computer Science and Communication Department

In partial fulfillment of the requirements for the degree of
LICENCE IN COMPUTER SCIENCE

By

Kacem Aymen & Hosni Rayen

IMPLEMENTATION OF A BIG DATA ARCHITECTURE FOR NATIVE ANALYTICS ON FAST AND CHANGING DATA

Defended on 13 June 2024, before the jury composed of:

Mr. Hammami Mohamed
Ms. Krichen Fatma
Mr. Ben Aouicha Mohamed
Mr. Abida Ghazi

President
Rapporteur
Academic Advisor
Professional Advisor

Academic Year : 2023 - 2024

LSI3

Dedication

To my beloved parents, Ridha and Salma, thank you for your unwavering support and encouragement throughout my education. Your sacrifices and love have guided me at every step, and I am forever grateful.

To my dear friends, thank you for your endless support and friendship. Your encouragement and companionship have made every step of this journey more enjoyable and meaningful. Whether in moments of joy or times of difficulty, you have always been there. I am deeply grateful for the laughter, the shared experiences, and the unwavering support. Your friendship has been a true blessing in my life.

To my dear partner Rayen, you have been more than just a friend. You've been my brother, sharing in both the triumphs and the challenges. Thank you for your constant support and for being a source of strength and inspiration over the years. Without you, this work would not have been possible.

Aymen Kacem

Dedication

To my father Sahbi and my mother Lilia No words can fully convey my deep appreciation and gratitude. Your support and presence have been crucial in overcoming various challenges. I hope this work makes you proud of your son.

To my friends For all the unforgettable moments we have shared, and for their support and friendship. I dedicate this modest work to them as a sign of my appreciation and gratitude.

To my dear partner Aymen Without your support, kindness, and encouragement, this work would not have been possible. Let this work stand as a testament to my gratitude and sincere friendship.

Rayen Hosni

Acknowledgment

We are profoundly grateful to our academic advisor, **Ben Aouicha Mohamed**, for his insightful guidance and unwavering support throughout the course of this project. Our deepest thanks also go to our professional advisor, **Abida Ghazi**, whose practical advice and expertise were crucial to the successful implementation of this project.

We extend special thanks to the **Data Engineering and Search Unit (DES)** for their invaluable contributions and support. Their expertise in data engineering has been instrumental in guiding us through complex data-related challenges.

We would also like to acknowledge **Sifast**, the company that provided us with professional guidance. Their commitment to innovation and excellence greatly inspired us and played a significant role in the success of our project.

We extend our heartfelt gratitude to the esteemed members of the jury for their interest in our project, their willingness to review it, and their valuable evaluation.

We would like to thank the **Faculty of Sciences of Sfax** and **the Computer Science and Communication Department** for providing us with the necessary resources and a conducive environment for our research. We also express our gratitude to all our teachers for the time and effort they invested in us, ensuring a high-quality education throughout our years of study.

Lastly, we sincerely appreciate the unwavering support and encouragement from our families and friends, without which the completion of this project would not have been possible.

List of Abbreviations and Terms

FSS	Faculty of Sciences of Sfax
HDFS	Hadoop Distributed File System
YARN	Yet Another Resource Negotiator
MPP	Massive Parallel Processing
RDD	Resilient Distributed Datasets
API	Application Programming Interface
VM	Virtual Machine
SSD	solid-state drive
UI	User Interface
Hue	Hadoop User Experience

Table of Contents

List of Figures	7
List of Tables	9
General Introduction	10
Chapter 1: Project Context	11
1 Real-time Analytics	11
1.1 Industry Trends and Data Growth	12
1.2 Real-Time Analytics Use Cases	12
2 Traditional and modern architectures	13
2.1 Traditional architectures	13
2.2 Modern architecture	13
2.3 Example (Xiaomi architecture)	13
3 Project Relevance	15
4 Objectives Alignment	15
5 Conclusion	15
Chapter 2: Design and Architecture	16
1 Ingestion Layer (Apache Kafka)	17
2 Batch Layer (Apache Hadoop and Spark)	18
2.1 Apache Hadoop	18
2.2 Erasure Coding	19
2.3 Apache Spark	20
3 Streaming Layer (Apache Kudu and Spark Streaming)	21
3.1 Apache Kudu	21
3.2 Apache Spark Streaming	22
4 Serving Layer (Apache Impala)	23
5 Kudu and Impala	24
6 Conclusion	24
Chapter 3: Configuration and Deployment	25
1 Deployment Environment	25
2 Apache Kafka	26
3 Apache Hadoop	32
4 Kafka-HDFS Connector	34

5	Apache Spark	37
6	Apache Kudu and Impala Integration	42
6.1	Apache Kudu architecture and components	42
6.2	Apache Kudu Setup	43
6.3	Apache Impala Architecture and Components	45
6.4	Apache Impala Setup	46
6.5	Apache Kudu with Apache Impala	47
6.6	Hadoop User Experience	49
7	Web Interfaces for Monitoring and Management	49
Chapter 4: Uses case		54
1	Introduction	54
2	Scenario Definition	54
3	Demonstration	54
3.1	Data Ingestion	55
3.2	Streaming	55
3.3	Batch	61
4	Conclusion	63
General Conclusion		64
Abstract		65
Web Bibliography		66

List of Figures

1	Real-time Analytics [1]	11
2	Big Data Market Size and trends 2023-2030 [2]	12
3	old Xiaomi Architecture [3]	14
4	new Xiaomi Architecture [3]	14
5	Big Data Architecture	16
6	Apache Kafka	17
7	Apache Hadoop	18
8	XOR Erasure Coding [8]	19
9	Apache Spark	20
10	Apache Kudu	21
11	Apache Spark Streaming	22
12	Apache Impala	23
13	Apache Kafka Architecture [16]	26
14	Zookeeper status output	28
15	Kafka status output	28
16	Apache Hadoop Architecture [17]	32
17	Apache Spark Architecture [19]	37
18	Apache Kudu Network Architecture [10]	42
19	Apache Impala Architecture [20]	45
20	Kudu with Impala Architecture [21]	48
21	NameNode UI	50
22	YARN UI	51
23	Spark UI	51
24	Kudu UI	52
25	Impala UI	52
26	Hue UI	53
27	Producer Output	55
28	Creating kudu table using impala-shell	56
29	Spark Streaming job terminal	57
30	Spark Active Job	58
31	Spark Completed Jobs	58
32	Select Query	59
33	Impala UI Queries	60
34	Hue Browser Table Result	60
35	Hue browser Graph Result	61

36	HDFS Connector output	61
37	HDFS number of blocks	62
38	Spark Batch Output	62
39	Querying with Spark	63

List of Tables

1	Comparison between Apache Kafka and Apache NiFi [4] [5]	17
2	Comparison between Apache Spark and Hadoop MapReduce [9] [6] . . .	20
3	Comparison between Apache Kudu and Apache HBase [10] [11]	21
4	Comparison between Apache Spark Streaming and Apache Storm [12] [13]	22
5	Comparison between Apache Impala and Apache Drill [14] [15]	23

General Introduction

Today, businesses and organizations are swimming in data. Every day, they collect huge amounts of information from various sources. But having lots of data isn't useful unless you can quickly make sense of it. That's where our project, "Implementation of a Big Data Architecture for Native Analytics on Fast and Changing Data," comes into play. We're building a system that can handle and analyze big piles of data quickly and efficiently.

At the heart of our project are two key technologies: Apache Impala and Apache Kudu. Think of Apache Impala as a super-fast librarian who can find the book you need in a massive library in seconds. Apache Kudu, on the other hand, is like a magical bookshelf that can change and move around books quickly to keep everything organized. Together, they're a powerful team that can help us get answers from our data faster than ever before.

This project isn't just about putting two technologies together. It's about making it easier for businesses to make smart decisions quickly. Whether it's a bank trying to understand market trends or a hospital wanting to improve patient care, our system is designed to help them get the information they need when they need it.

We're also keeping an eye on the future. The way we're building this system will make it easier to update and improve as new technologies come along. This means that our project is not just solving today's problems but is also ready for tomorrow's challenges.

In short, our project is all about making big data easy to handle and quick to understand. By using Apache Impala and Apache Kudu, we're building a solution that will help businesses stay ahead in a world where data is king. We're excited to take on this challenge and open up new possibilities for using data in smart and fast ways.

Chapter 1: Project Context

In this chapter, we explore the imperative for advanced big data architectures amidst the surge of digital transformation across industries. As data grows exponentially in volume, velocity, and variety, the limitations of traditional data processing systems become increasingly apparent. We discuss the challenges posed by this new data landscape, highlight the potential of technological advancements like Apache Impala and Apache Kudu, and emphasize the project's relevance in addressing these challenges. This chapter aims to illustrate the critical need for scalable, efficient, and resilient big data solutions.

1 Real-time Analytics

Real-time analytics refers to the process of analyzing data as it is generated or received, enabling immediate insights and actions. This capability is crucial in today's fast-paced world where timely decision-making can significantly impact business outcomes. By leveraging real-time analytics, organizations can monitor and respond to events as they happen, rather than relying on historical data analysis. This approach is particularly valuable in industries such as finance, healthcare, retail, and telecommunications, where rapid data processing can enhance fraud detection, improve patient care, optimize inventory management, and ensure network reliability[1].

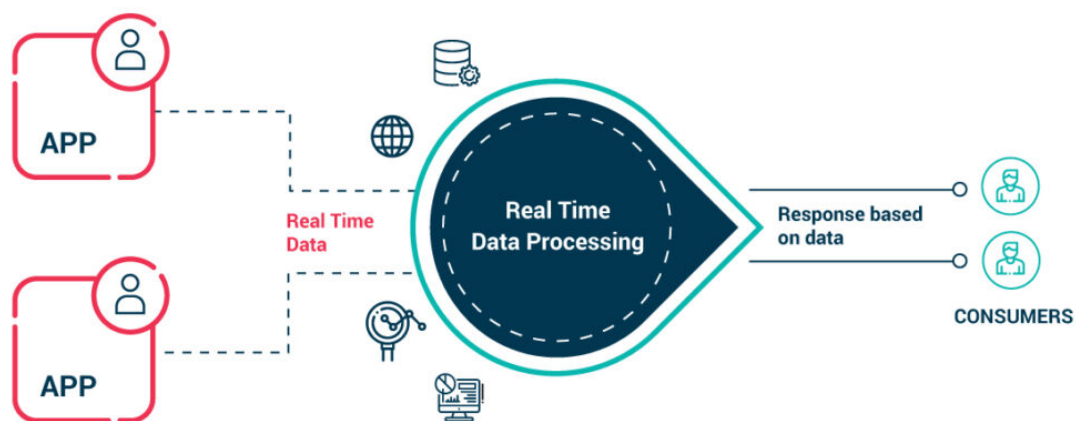


Figure 1. Real-time Analytics [1]

1.1 Industry Trends and Data Growth

The digital transformation across various sectors has led to an unprecedented increase in data generation. This exponential growth in data volume, velocity, and variety, presents both opportunities and challenges. Industries such as finance, healthcare, e-commerce, and IoT are generating petabytes of data daily, necessitating advanced analytics solutions to extract valuable insights in real-time [2].

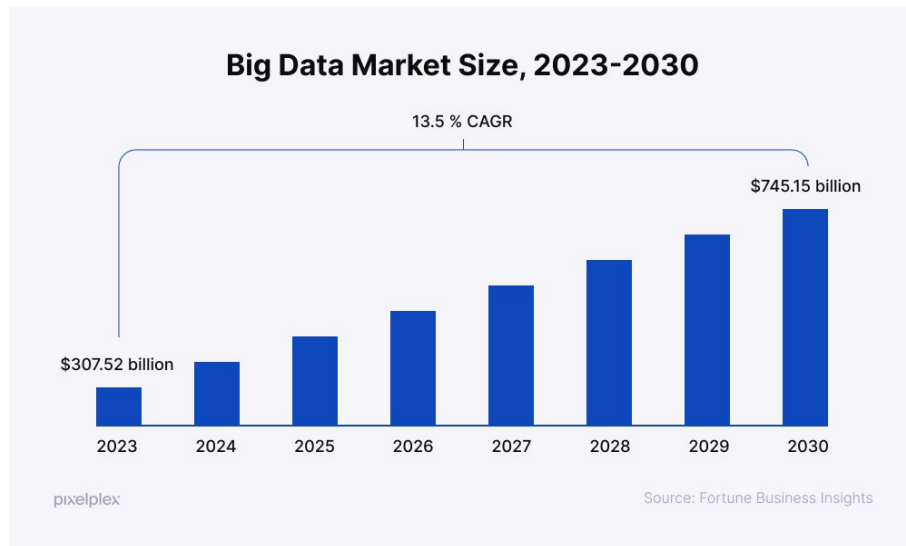


Figure 2. Big Data Market Size and trends 2023-2030 [2]

1.2 Real-Time Analytics Use Cases

Real-time analytics is a game-changer for many industries, enabling them to make immediate, data-driven decisions. Here are some key use cases:

- **Financial Services:** In the financial sector, real-time analytics is used for fraud detection, risk management, and algorithmic trading.
- **Healthcare:** Real-time analytics in healthcare can improve patient outcomes by enabling immediate data analysis from various sources such as patient monitors, wearable devices, and electronic health records.
- **Internet of Things (IoT):** In IoT applications, real-time analytics is crucial for monitoring and managing connected devices.
- **Telecommunications:** Telecommunications companies use real-time analytics to monitor network performance and customer experience.

2 Traditional and modern architectures

2.1 Traditional architectures

The older systems are increasingly unable to meet the demands of the fast-paced environment. They struggle with processing large volumes of data swiftly and adapting to new requirements, which leads to sluggish insights and decision-making processes. A major challenge is the lack of scalability in existing systems, which cannot efficiently process and analyze the burgeoning amounts of data, resulting in bottlenecks and delayed insights. Additionally, these systems face issues with latency in data processing, making it difficult to achieve the real-time analytics that modern applications require. Furthermore, the diversity of data sources and formats introduces complexity in integration and analysis, necessitating more flexible and dynamic processing capabilities to handle data heterogeneity effectively.

2.2 Modern architecture

New real-time architectures are making a big difference. These use modern technologies like Apache Impala and Apache Kudu to work with data. Apache Impala is an open-source SQL query engine that allows for lightning-fast reading and writing, making it ideal for real-time querying on large datasets. Apache Kudu, on the other hand, is a storage system that bridges the gap between HDFS and HBase, offering both fast inserts and updates alongside efficient columnar scans, which is crucial for real-time analytics on fast-changing data. Together, these technologies enable organizations to achieve high performance, efficiency, and scalability in their data processing and storage operations, facilitating more dynamic and responsive data-driven decision-making.

2.3 Example (Xiaomi architecture)

Traditional architecture

Before switching to Apache Kudu and Impala, Xiaomi's old data system struggled with managing the huge amount of data from its products and services, storing over 20 billion records daily using HBase and a separate Sequence file database. The data processing, involving Apache Hive, Hadoop MapReduce, and Spark, was slow and complex, taking up to a day for analysis results, too slow for today's fast-paced business needs. Additionally, the system had issues with the order of log entries, often requiring data from past periods to complete analyses, further delaying insights [3].

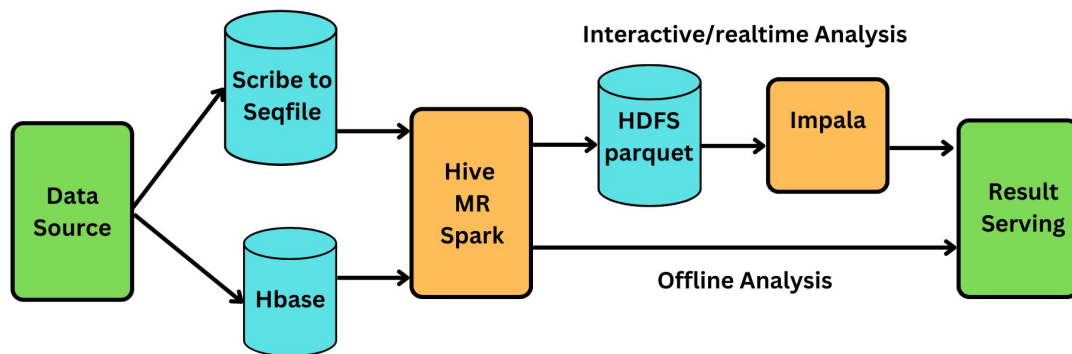


Figure 3. old Xiaomi Architecture [3]

Modern architecture

Xiaomi upgraded its data system by adopting an ETL pipeline with Apache Kudu, Kafka, and Storm, greatly enhancing the efficiency and speed of analyzing mobile and server service data. This setup uses Kafka for data aggregation, Storm for real-time processing, and Kudu for storing both raw and processed data. Impala facilitates quick SQL-based data access, while BI services can directly retrieve data from Kudu or through Impala. This new architecture significantly improved performance and reliability, simplifying data processing with Kudu enabling fast, interactive analysis with minimal latency[3].

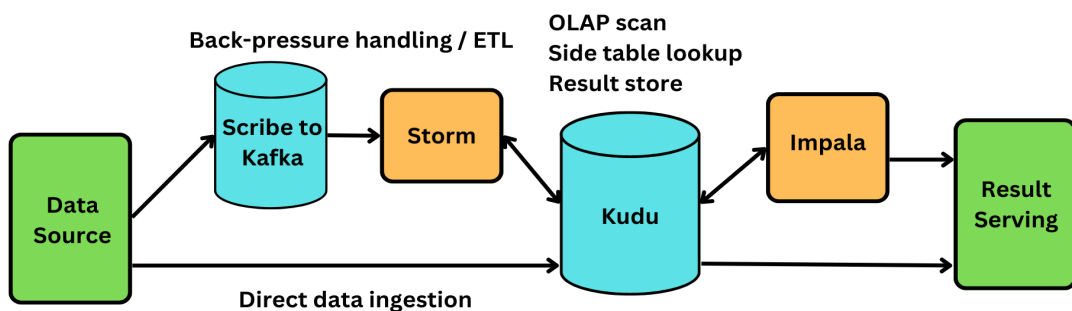


Figure 4. new Xiaomi Architecture [3]

3 Project Relevance

This project holds immense relevance across industries grappling with the intricacies of data management. By streamlining operations and facilitating agile decision-making, it stands as a beacon of efficiency in today's fast-paced business landscape. The project's significance transcends boundaries, offering invaluable solutions to businesses navigating the complexities of data utilization. As industries evolve and data volumes soar, the project's adaptive capabilities ensure seamless scalability, empowering organizations to navigate the data deluge with confidence. Its impact extends far beyond mere operational enhancements, catalyzing a fundamental shift in how businesses perceive and leverage data assets. In essence, this project is not just about technological advancement; it represents a paradigm shift in industry practices, heralding a new era of data-driven innovation and strategic foresight.

4 Objectives Alignment

The primary objectives of this project are to:

1. Design a scalable big data architecture incorporating Apache Impala for analytics and Apache Kudu for data storage.
2. Implement a configuration strategy that ensures fault tolerance and efficient data storage, leveraging erasure coding in Hadoop.
3. Document the configuration, deployment, and integration processes in a comprehensive guide.
4. Conduct performance benchmarking to demonstrate the architecture's effectiveness in delivering real-time analytics.

5 Conclusion

The development of advanced big data architectures is crucial in today's data-driven era. This project is a vital step toward enhancing our capabilities in big data analytics, laying a strong foundation for future advancements. By adopting innovative technologies, it sets the stage for significant improvements in decision-making and operational efficiencies across various industries, ensuring our data architecture can adapt to future challenges.

Chapter 2: Proposed Architecture

The architecture upholds Lambda architecture principles, ensuring scalability and fault tolerance while adeptly handling large data volumes. It incorporates four critical components: the ingestion layer, the batch layer, the streaming layer, and the serving layer, each chosen for its distinct advantages over comparable technologies.

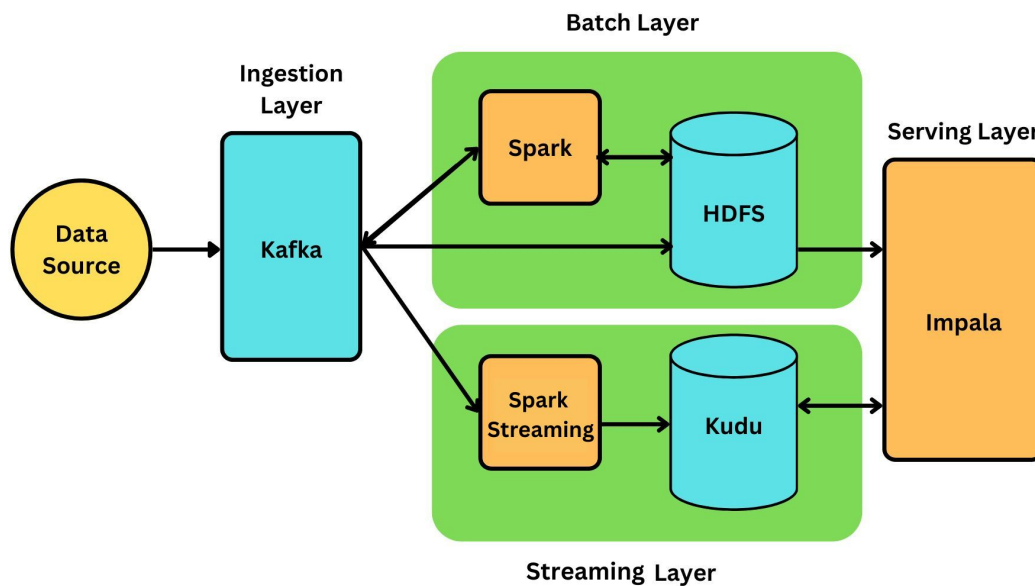


Figure 5. Big Data Architecture

Data from various sources is ingested through Apache Kafka, which acts as a high-throughput, distributed messaging system. The ingested data is then processed in two distinct layers: the Batch Layer and the Streaming Layer. In the Batch Layer, data is stored in HDFS and processed using Apache Spark, managed by YARN for resource allocation. The Streaming Layer, designed for real-time data processing, now exclusively uses Spark Streaming, to handle real-time data streams efficiently. Apache Kudu may also be integrated into this layer for fast analytics on fast data. The processed data from both layers is then made queryable through Apache Impala in the Serving Layer, which provides massive parallel processing (MPP) SQL query engine capabilities. This setup is ideal for environments where data processing speed and efficiency are prioritized over visual data representation.

1 Ingestion Layer (Apache Kafka)

Apache Kafka is an open-source stream-processing platform designed for high-volume, real-time data handling. Developed by LinkedIn and contributed to the Apache Software Foundation, it excels in processing and moving large data streams efficiently between applications or systems [4]. Apache Kafka is selected for its exceptional performance in data ingestion, particularly when compared to alternatives like Apache NiFi.



Figure 6. Apache Kafka

Kafka vs NiFi

To highlight the differences and strengths of Apache Kafka compared to Apache NiFi, we present the following table:

Feature	Apache Kafka	Apache NiFi
Primary Use	High-throughput, low-latency messaging system	Dataflow management system
Core Strengths	- High throughput - Scalable - Durable messaging	- Highly configurable and versatile data routing
Scalability	Excellent, can handle millions of messages per second	Good, but more manual and complex to scale
Ease of Use	Requires understanding of distributed systems	Graphical interface simplifies management
Data Transformation	Limited to basic functions	Extensive and customizable processing capabilities
Integration	Broad integration with big data technologies	Wide range of processors for various data sources and sinks
Fault Tolerance	High, with replication and partitioning	Good, supports data buffering and backpressure
Use Case Examples	- Real-time analytics - Event sourcing - Log aggregation	- Complex data collection - Data routing with transformation

Table 1. Comparison between Apache Kafka and Apache NiFi [4] [5]

2 Batch Layer (Apache Hadoop and Spark)

2.1 Apache Hadoop

Apache Hadoop is a comprehensive, open-source framework designed for scalable, distributed computing. It enables the processing and storage of large datasets across clusters of computers using simple programming models. Hadoop is comprised of several modules, each fulfilling a critical role in its ecosystem [6].



Figure 7. Apache Hadoop

Hadoop Distributed File System:

The Hadoop Distributed File System (HDFS) is a fundamental component designed for storing vast amounts of data reliably across a distributed network of servers. It shines in archiving raw data, offering a scalable and robust solution for managing extensive datasets. By replicating data blocks across multiple nodes, HDFS achieves high availability and fault tolerance, meeting the demands of large-scale batch processing tasks efficiently.

Yet Another Resource Negotiator:

Yet Another Resource Negotiator (YARN) is another critical component of Apache Hadoop that manages and schedules resources across the cluster. YARN enhances the system's processing capabilities by allowing multiple data processing engines, such as MapReduce and Spark, to handle data stored in HDFS. It optimizes resource utilization and provides a flexible platform for executing a wide variety of data processing tasks, contributing significantly to Hadoop's versatility and efficiency in handling big data challenges.

2.2 Erasure Coding

Erasure coding is a method used in distributed storage systems to provide fault tolerance while minimizing storage overhead. Unlike traditional replication, which typically triples the storage requirements, erasure coding achieves redundancy with much lower overhead, making it an efficient solution for large-scale data storage [7].

XOR-Based Erasure Codes

XOR-based erasure codes are a simple yet effective method for data redundancy in distributed storage systems. They work by dividing data into k data blocks and generating t parity blocks using the XOR operation. This allows the original data to be reconstructed from any k out of the $k+t$ blocks.

For example, in an XOR(6, 3) scheme:

- $k=6$: Number of data blocks.
- $t=3$: Number of parity blocks.
- The system can tolerate up to 3 simultaneous failures and still recover the original data.

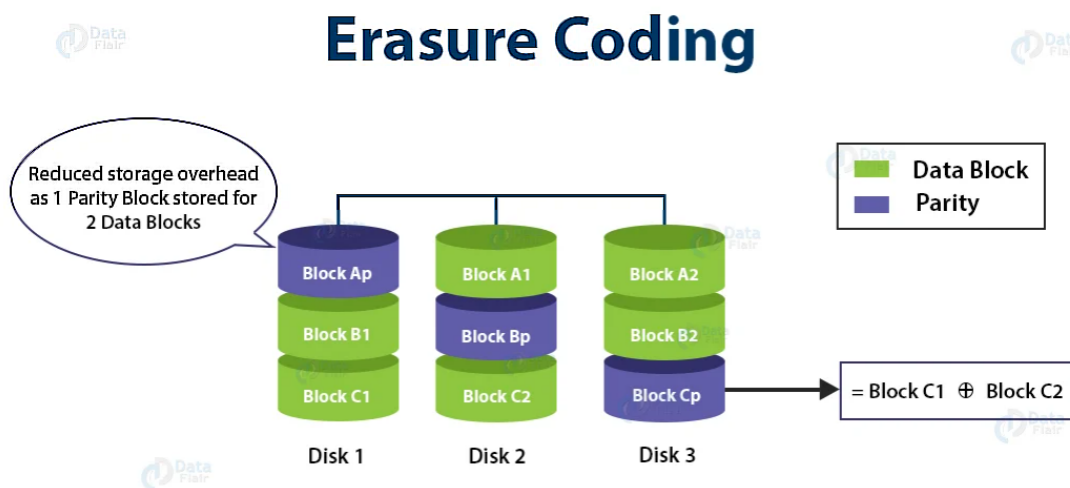


Figure 8. XOR Erasure Coding [8]

2.3 Apache Spark

Apache Spark excels in batch processing by leveraging in-memory computing, significantly speeding up data analysis over traditional disk-based methods. Its comprehensive APIs across Python, Java, Scala, and R facilitate the development of complex processing tasks. Spark's scalability and fault tolerance make it a top choice for efficient batch processing in various fields [9].



Figure 9. Apache Spark

Spark vs MapReduce

The following table provides a side-by-side comparison between Apache Spark and Hadoop MapReduce, shedding light on their distinct features, performance metrics, and suitable use cases, thereby underlining the scenarios where Spark's in-memory processing advantage becomes particularly evident.

Feature	Apache Spark	Hadoop MapReduce
Primary Use	General-purpose distributed data processing engine that supports batch, interactive, iterative, and streaming data processing	Batch processing framework designed for large-scale data processing in a distributed computing environment
Core Strengths	- Fast processing speed due to in-memory computation - Supports real-time processing and machine learning - Easy to use APIs for complex transformations	- Highly scalable and robust framework for processing huge datasets - Good for linearly scalable, high latency batch jobs
Performance	Significantly faster than MapReduce due to in-memory processing	Slower, disk-based processing suitable for straightforward batch processing tasks
Fault Tolerance	Fault tolerance through lineage information and RDDs (Resilient Distributed Datasets)	Fault tolerance achieved through data replication and task re-execution upon failure
Use Case Examples	- Real-time analytics - Machine learning models - Graph processing	- Large scale processing jobs - Batch jobs that do not require immediate results

Table 2. Comparison between Apache Spark and Hadoop MapReduce [9] [6]

3 Streaming Layer (Apache Kudu and Spark Streaming)

3.1 Apache Kudu

Apache Kudu plays a critical role in the speed layer of our big data architecture, particularly for scenarios requiring fast data updates and real-time analytics. Kudu combines the best features of Hadoop's HDFS and HBase, offering a simplified data model that supports both fast inserts and updates. This makes it particularly useful for real-time applications where speed and efficiency are paramount [10].



Figure 10. Apache Kudu

Kudu vs HBase

Kudu is often preferred over HBase for certain applications due to its simplicity and performance characteristics. Below is a comparative table highlighting the differences between Kudu and HBase:

Feature	Apache Kudu	Apache HBase
Data Model	Simplified data model, supporting fast data scans and updates.	Complex data model with manual sharding and region management.
Performance	Optimized for fast inserts and updates, making it ideal for real-time analytics.	Optimized for high read/write speeds but requires careful management of regions for optimal performance.
Ease of Use	Easier to manage due to automatic handling of data partitioning and recovery.	Requires more manual intervention to manage regions and ensure scalability.
Use Cases	Real-time analytics, time series data, and fast data modifications.	Large-scale batch processing and when strong consistency is required.

Table 3. Comparison between Apache Kudu and Apache HBase [10] [11]

3.2 Apache Spark Streaming

Apache Spark Streaming excels in real-time data processing by leveraging Spark's powerful in-memory computing framework, allowing for rapid and efficient analysis of streaming data. Its scalability ensures it can handle large volumes of live data, while its built-in fault tolerance guarantees reliable processing. This makes Spark Streaming an ideal solution for industries requiring real-time analytics [12].



Figure 11. Apache Spark Streaming

Spark Streaming vs Apache Storm

The following table provides a side-by-side comparison between Apache Spark Streaming and Apache Storm, highlighting their key features and use cases.

Feature	Apache Spark Streaming	Apache Storm
Primary Use	Real-time stream processing framework integrated with Spark for batch processing	Real-time stream processing framework
Core Strengths	- Micro-batching for fault tolerance and exactly-once processing semantics - Integration with Spark for unified batch and stream processing	- Low latency, real-time stream processing - Fault tolerance through record acknowledgments and replay mechanisms
Performance	Higher latency compared to Storm, but better suited for complex computations and integration with Spark ecosystem	Very low latency, suitable for time-sensitive processing
Fault Tolerance	Fault tolerance through Spark's fault recovery mechanisms	Fault tolerance through acking and replay of messages
Use Case Examples	- Real-time analytics integrated with batch processing - Complex event processing - Machine learning on real-time data	- Internet of Things (IoT) data processing - Fraud detection - Real-time analytics

Table 4. Comparison between Apache Spark Streaming and Apache Storm [12] [13]

4 Serving Layer (Apache Impala)

Apache Impala is a high-performance, open-source SQL query engine for Hadoop, enhanced by integration with Apache Kudu for real-time querying on large datasets. This combination offers fast analytics on changing data, making it ideal for immediate insights crucial for decision-making. Impala and Kudu together provide a superior solution for real-time data analysis, surpassing alternatives in speed and efficiency [14].



Figure 12. Apache Impala

Apache Impala vs Apache Drill

To further understand the strengths of Apache Impala within this architecture, a comparative analysis with Apache Drill, another prominent SQL query engine, is presented below. This comparison highlights the distinct features and optimal use cases for each technology.

Feature	Apache Impala	Apache Drill
Query Execution	MPP SQL Engine optimized for Hadoop	Distributed SQL Query Engine that supports a variety of data sources
Data Storage Compatibility	Primarily HDFS, Apache Kudu, and HBase	Supports a wide variety of NoSQL databases and file systems
SQL Capability	Full SQL support for structured data	Full SQL support, particularly effective on non-relational data
Performance	High performance on large clusters, ideal for BI applications	Highly flexible, supports schema-free querying, suitable for ad-hoc analysis across different data formats
Use Case	Optimized for business intelligence and analytics on structured data within Hadoop environments	Best suited for exploratory and ad-hoc analysis across diverse data formats and sources

Table 5. Comparison between Apache Impala and Apache Drill [14] [15]

5 Kudu and Impala

The integration with Apache Kudu further amplifies Impala's strengths, offering a powerful combination for real-time analytics. Kudu's architecture is designed for fast analytics on rapidly changing data, which complements Impala's real-time querying capabilities. Here's why this integration is particularly beneficial:

- **Seamless Querying:** Direct querying on Kudu tables allows for real-time analytics without the need for data movement or transformation, enabling immediate insights.
- **Optimized Storage and Access:** Combining Impala's querying prowess with Kudu's fast data inserts and updates supports swift SQL querying, outperforming other systems in speed and efficiency.
- **Simplified Data Architecture:** This integration streamlines the data architecture, reducing complexity and maintenance, thus focusing more on analysis rather than data management.

In essence, the Impala-Kudu integration offers a streamlined, efficient path for real-time querying and analytics. This integration is particularly effective for use cases requiring immediate data analysis and decision-making capabilities. Impala, with its integration with Kudu, provides a more focused and efficient solution for real-time analytics within Hadoop environments. This makes it an ideal choice for organizations prioritizing speed and efficiency in their data analysis operations, significantly outshining alternatives in scenarios where real-time performance and data freshness are paramount.

6 Conclusion

In conclusion by integrating Apache Kafka for data ingestion, Apache Hadoop and Spark for batch processing, and Apache Kudu and Spark Streaming for real-time processing, the architecture ensures robust and responsive data processing capabilities. The serving layer, utilizing Apache Impala, provides high-performance SQL querying, enabling real-time analytics and decision-making. This comprehensive architecture addresses the limitations of traditional systems, offering a modern solution for managing fast and changing data effectively.

Chapter 3: Configuration and Deployment

1 Deployment Environment

The successful deployment of our big data architecture hinges on a carefully selected set of hardware and software prerequisites designed to ensure optimal performance, scalability, and reliability. For our project, we utilized a total of six virtual machines (VMs). Three of these VMs are equipped with 16 GB RAM and 500 GB of solid-state drive (SSD) storage, while the other three VMs have 32 GB RAM and 500 GB SSD storage.

The VMs with 16 GB RAM are dedicated to our Hadoop cluster. Specifically, one VM serves as the namenode, and the remaining two VMs function as datanodes. Additionally, the namenode machine also hosts a Kafka broker and Spark, optimizing resource utilization and facilitating seamless data processing and management within the cluster.

The three VMs with 32 GB RAM are allocated for more intensive processing tasks. One of these VMs is dedicated to Apache Impala and Spark Streaming, providing robust query execution and real-time data processing capabilities. The remaining two VMs are used as Kudu tablet servers and data node, ensuring efficient and scalable storage and retrieval of data.

On the software front, our architecture is built upon the Ubuntu 22.04 operating system. We have deployed the core components, including Apache Kafka, Apache Hadoop, Apache Spark, Apache Kudu, and Apache Impala, using their latest stable versions to take advantage of recent improvements and security patches.

By meeting these system requirements, we have laid a solid foundation for our architecture, enabling it to efficiently process, store, and analyze big data at scale. This setup ensures that our big data solution is robust, scalable, and ready to handle the demands of large datasets with high throughput and low latency.

2 Apache Kafka

Apache Kafka is a distributed streaming platform that consists of several key components:

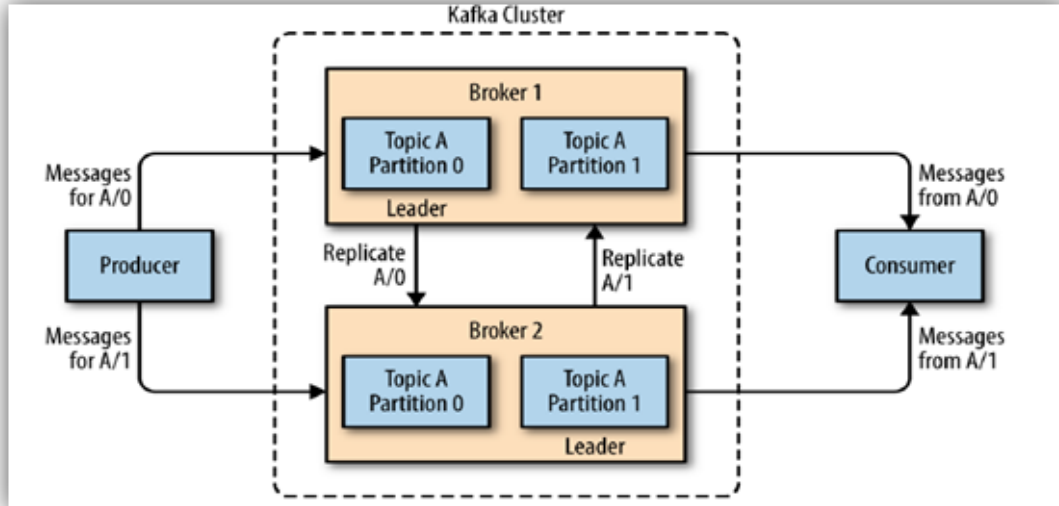


Figure 13. Apache Kafka Architecture [16]

- **Producers:** Applications that send data to Kafka.
- **Consumers:** Applications that read data from Kafka.
- **Brokers:** Servers that store data and serve client requests.
- **Topics:** Categories to which records are published.
- **Partitions:** Sub-divisions of topics to allow parallel processing.
- **ZooKeeper:** Coordinates and manages Kafka brokers.

Installation

To install Apache Kafka, we first need to install Java, as Kafka is built on top of the Java Virtual Machine (JVM). It's recommended to use OpenJDK version 11.

```
sudo apt update
sudo apt install openjdk-11-jdk -y
java -version
```

After installing Java, download the latest version of Apache Kafka from the official website. As of writing, the latest stable version is 3.7.0, but we should check the Apache Kafka website for the most recent version. Save the download to the '/home/user' directory.

```
cd /home/user
wget
https://downloads.apache.org/kafka/3.7.0/kafka_2.12-3.7.0.tgz
tar -xvf kafka_2.12-3.7.0.tgz
cd kafka_2.12-3.7.0
```

Create Kafka and ZooKeeper Systemd Unit files

```
sudo nano /etc/systemd/system/zookeeper.service
```

```
[Unit]
Description=Apache Zookeeper server
Documentation=http://zookeeper.apache.org
Requires=network.target remote-fs.target
After=network.target remote-fs.target

[Service]
Type=simple
ExecStart=/home/user/kafka/bin/zookeeper-server-start.sh
/home/user/kafka/config/zookeeper.properties
ExecStop=/home/user/kafka/bin/zookeeper-server-stop.sh
Restart=on-abnormal

[Install]
WantedBy=multi-user.target
```

Save the changes and exit.

Next, create Kafka's systemd file

```
sudo nano /etc/systemd/system/kafka.service
```

```
[Unit]
Description=Apache Kafka Server
Documentation=http://kafka.apache.org/documentation.html
```

```
Requires=zookeeper.service
```

```
[Service]
```

```
Type=simple
```

```
Environment="JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64"
```

```
ExecStart=/home/user/kafka/bin/kafka-server-start.sh
```

```
/home/user/kafka/config/server.properties
```

```
ExecStop=/home/user/kafka/bin/kafka-server-stop.sh
```

```
[Install]
```

```
WantedBy=multi-user.target
```

Start Kafka and ZooKeeper Systemd Services

```
sudo systemctl daemon-reload
```

```
sudo systemctl start zookeeper
```

```
sudo systemctl start kafka
```

Check Kafka and Zookeeper services

```
sudo systemctl status zookeeper
```

```
user@uservm:~$ sudo systemctl status zookeeper
● zookeeper.service - Apache Zookeeper server
   Loaded: loaded (/etc/systemd/system/zookeeper.service; disabled; vendor preset: enabled)
   Active: active (running) since Fri 2024-05-17 10:37:35 CET; 3 days ago
     Docs: http://zookeeper.apache.org
    Main PID: 2843 (java)
      Tasks: 52 (limit: 38415)
    Memory: 420.6M
       CPU: 12min 11.225s
    CGroup: /system.slice/zookeeper.service
            └─2843 iava -Xmx512M -Xms512M -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -XX:InitiatinoHe
```

Figure 14. Zookeeper status output

```
sudo systemctl status kafka
```

```
● kafka.service - Apache Kafka Server
   Loaded: loaded (/etc/systemd/system/kafka.service; disabled; vendor preset: enabled)
   Active: active (running) since Fri 2024-05-17 10:37:40 CET; 3 days ago
     Docs: http://kafka.apache.org/documentation.html
    Main PID: 3266 (java)
      Tasks: 82 (limit: 38415)
    Memory: 1.0G
       CPU: 27min 26.802s
    CGroup: /system.slice/kafka.service
            └─3266 /usr/lib/jvm/java-11-openjdk-amd64/bin/java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPauseMillis=20 -XX:InitiatinoHe
```

Figure 15. Kafka status output

Configuration

Topic creation:

Once Kafka is installed, we need to configure it to create topics, and then produce and consume messages. Here's how to create a topic named "test":

```
bin/kafka-topics.sh --create --topic test --bootstrap-server \
localhost:9092 --replication-factor 1 --partitions 1
```

This command creates a topic named "test" with one partition and one replication factor.

Producer script:

To send messages to the topic, use the Kafka producer script:

```
bin/kafka-console-producer.sh --topic test \
--bootstrap-server localhost:9092
```

Once the producer script is running, every line we enter into the terminal will be sent as a message to the "test" topic. we can type our messages and press Enter to send them.

Consumer script:

To read the messages from the topic, we use the Kafka consumer script:

```
bin/kafka-console-consumer.sh --topic test --from-beginning \
--bootstrap-server localhost:9092
```

This command will display all messages sent to the "test" topic right from the beginning.

These basic commands get us started with producing and consuming messages in Kafka. For production environments, we will need to dive deeper into Kafka's configuration options to optimize performance, durability, and security.

Custom Producer Script:

In our project, we used a custom Kafka producer script to simulate stock market data and send it to a Kafka topic named "market". Here is the script we used:

```
import sys
from time import sleep
from json import dumps
from kafka import KafkaProducer
import random
import datetime
import threading

# Function to generate simulated stock market data
def generate_stock_data(producer):
    stock_symbols = ['AAPL', 'GOOGL', 'MSFT', 'AMZN', 'FB',
                     'TSLA', 'NFLX', 'NVDA', 'INTC', 'AMD',
                     'IBM', 'CSCO', 'ORCL', 'QCOM', 'PYPL',
                     'ADBE', 'CRM', 'SQ', 'UBER', 'LYFT',
                     'BABA', 'JD', 'TSM', 'SNE', 'NIO',
                     'RKT', 'GM', 'F', 'GE', 'BA',
                     'PFE', 'JNJ', 'MRNA', 'ABT', 'GILD',
                     'MRK', 'BNTX', 'BMY', 'TGT', 'WMT',
                     'HD', 'LOW', 'MCD', 'SBUX', 'YUM', 'KO',
                     'PEP', 'DPZ', 'MNST', 'WBA']

    while True:
        for symbol in stock_symbols:
            price = round(random.uniform(50, 1000), 2)
            volume = random.randint(1000, 10000)
            high = round(price * random.uniform(1.01, 1.1), 2)
            low = round(price * random.uniform(0.9, 0.99), 2)
            open_price = round(random.uniform(low, high), 2)
            timestamp = datetime.datetime.now().strftime(
                ('%Y-%m-%d %H:%M:%S'))
            data = {
                'symbol': symbol,
                'price': price,
                'volume': volume,
```

```

        'high': high,
        'low': low,
        'open': open_price,
        'timestamp': timestamp
    }
    producer.send('market', value=data)
    print(f"Sent data: {data}")
    sleep(1)
    sys.stdout.flush()

# Kafka broker address
bootstrap_servers = ['localhost:9092']

# Create Kafka producer
producer = KafkaProducer(bootstrap_servers=bootstrap_servers,
                          value_serializer=lambda x:
                              dumps(x).encode('utf-8'))

# Start a separate thread for generating data
thread = threading.Thread(target=generate_stock_data,
                           args=(producer,))
thread.start()

# Keep the main thread running
try:
    while True:
        sleep(1)
except KeyboardInterrupt:
    print("Exiting...")
    producer.close()

```

This script uses the `KafkaProducer` class from the `kafka-python` library to create a Kafka producer. The `generate_stock_data` function generates random stock market data for a predefined list of stock symbols. It creates a JSON object containing the stock symbol, price, volume, high and low prices, open price, and a timestamp, then sends this data to the Kafka topic "market".

The data generation runs in a separate thread to allow continuous production of messages without blocking the main thread, which waits indefinitely until interrupted.

3 Apache Hadoop

Apache Hadoop is a framework that allows for the distributed processing of large data sets across clusters of computers using simple programming models. It consists of four main modules:

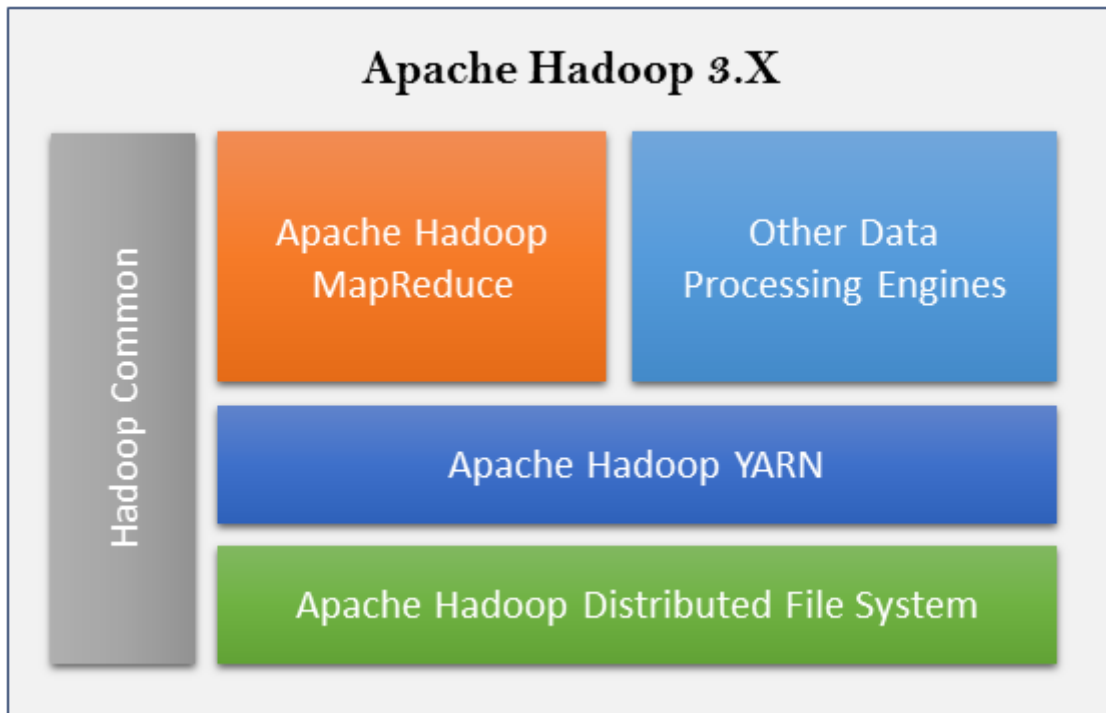


Figure 16. Apache Hadoop Architecture [17]

- **Apache HDFS (Hadoop Distributed File System):** A distributed file system designed to run on commodity hardware.
- **Apache YARN (Yet Another Resource Negotiator):** A resource management layer for scheduling and managing cluster resources.
- **Apache MapReduce:** A programming model for processing large data sets with a parallel, distributed algorithm.
- **Others:** Such as Apache Spark, Hive, HBase, Parquet, etc.

Installation

Apache Hadoop installation involves setting up Hadoop Distributed File System (HDFS) for scalable storage. Begin by ensuring Java is installed, as Hadoop is Java-based.

```
sudo apt update
sudo apt install openjdk-8-jdk
```

```
java -version
```

Download and extract Hadoop from the Apache website:

```
wget https://downloads.apache.org/hadoop/common
/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzvf hadoop-3.3.6.tar.gz
cd hadoop-3.3.6
```

Configuration

To configure a Hadoop cluster with 1 NameNode and 3 DataNodes, follow these steps:

1. **Edit `core-site.xml`:** Define the NameNode and HDFS properties.

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://NameNode:9000</value>
  </property>
</configuration>
```

2. **Edit `hdfs-site.xml`:** Define replication settings and enable erasure coding.

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.ec.policies.enabled</name>
    <value>XOR-2-1-1024k</value>
  </property>
</configuration>
```

3. **Edit `workers`:** Specify the DataNode hostnames.

```
DataNode1
DataNode2
DataNode3
```

Start Hadoop Services

On the NameNode:

```
bin/hdfs namenode -format  
sbin/start-dfs.sh
```

On each DataNode, ensure Hadoop is installed and configured similarly, then start the DataNode service:

```
sbin/hadoop-daemon.sh start datanode
```

Verify the Installation

Verify the installation by accessing Hadoop's web interface, typically available at `http://NameNode:9870`.

Additionally, check the status of the erasure coding policy:

```
hdfs erasurecode -listPolicies
```

This setup ensures a Hadoop cluster with 1 NameNode and 2 DataNodes, utilizing erasure coding for efficient data storage.

4 Kafka-HDFS Connector

The integration of Apache Kafka with Hadoop Distributed File System (HDFS) using Confluent's Kafka-HDFS connector allows us to stream data from Kafka topics directly into HDFS, enabling efficient storage and further processing using Hadoop ecosystem tools [18].

Downloading and Setting Up Confluent Platform

To set up the Kafka-HDFS connector, we need to download and install the Confluent Platform, which provides a comprehensive suite of tools for working with Kafka. Follow these steps:

1. **Download Confluent Platform:** Download the latest version of the Confluent Platform.

```
curl -O https://packages.confluent.io/archive/7.6/
confluent-community-7.6.1.tar.gz
```

2. **Extract the Archive:** Extract the downloaded archive to a suitable directory.

```
tar -xzf confluent-7.6.1.tar.gz
mv confluent-7.6.1 confluent
```

3. **Install the HDFS Connector:** This command installs the HDFS connector into our Confluent Platform.

```
confluent-hub install
confluentinc/kafka-connect-hdfs3:latest
```

Creating the HDFS Sink Connector Configuration

Next, we create a configuration file for the HDFS sink connector. This file specifies the properties required to connect Kafka to HDFS and manage the data flow. Below is the content of the configuration file named `hdfs-sink.properties`:

```
name=hdfs3-sink-connector
connector.class=io.confluent.connect.hdfs3
.Hdfs3SinkConnector
tasks.max=1
topics=market

# HDFS Configuration
hdfs.url=hdfs://NameNode:9000
hadoop.conf.dir=/home/user/hadoop/etc/hadoop/
hdfs.authentication.kerberos=false

# Output format and storage
format.class=io.confluent.connect.hdfs3.json.JsonFormat
flush.size=1000
rotate.interval.ms=6000
partitioner.class=io.confluent.connect.storage.partitionner
```

```
.DefaultPartitioner

# Schema configuration
value.converter=org.apache.kafka.connect.json.JsonConverter
value.converter.schemas.enable=false

# Internal storage for offset and status tracking
offset.storage.file.filename=/tmp/connect.offsets
offset.flush.interval.ms=10000
```

Running the HDFS Sink Connector

To run the HDFS sink connector, use the following command. This command starts the Kafka Connect standalone mode with the specified configuration files.

```
cd /home/user/confluent
./bin/connect-standalone
./etc/kafka/connect-standalone.properties
~/hdfs-sink.properties
```

This command initializes the Kafka Connect framework in standalone mode and uses the `hdfs-sink.properties` file to configure the HDFS sink connector. The connector will start streaming data from the specified Kafka topic (in this case, `market`) to HDFS.

Conclusion

Integrating Kafka with HDFS using Confluent's Kafka-HDFS connector provides a seamless way to stream data into HDFS for further processing and analysis. This setup leverages the strengths of both Kafka and HDFS, enabling real-time data ingestion and scalable storage. By following the steps outlined above, we can configure and run the HDFS sink connector to efficiently manage our data pipeline.

5 Apache Spark

Apache Spark is a unified analytics engine for large-scale data processing, with built-in modules for streaming, SQL, machine learning, and graph processing. Its core components include:

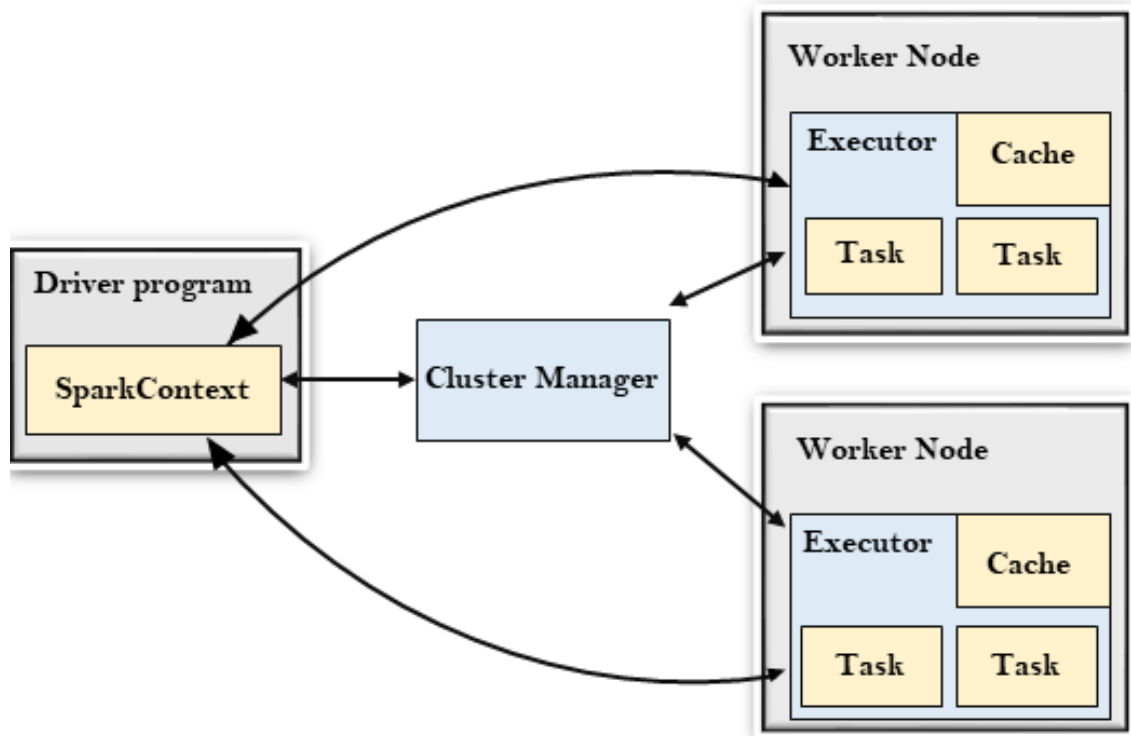


Figure 17. Apache Spark Architecture [19]

- **Driver Program:** The process that runs the main function of the application and creates the SparkContext.
- **Worker Node:** The nodes in the cluster that execute tasks and store data.
- **Cluster Manager:** The external service responsible for acquiring resources on the cluster (e.g., YARN, Mesos, or Kubernetes).
- **Spark Context:** The entry point to the Spark cluster, responsible for coordinating the execution of jobs.
- **Executor:** The distributed agent responsible for executing tasks on the worker nodes and returning the results to the driver program.

Installation

This section provides a step-by-step guide to download and install Apache Spark on an Ubuntu system.

```
sudo apt update
sudo apt install default-jdk scala
wget https://archive.apache.org/dist/spark/spark-3.5.1/
spark-3.5.1-bin-hadoop3.3.tgz
tar xvf spark-3.5.1-bin-hadoop3.3.tgz
```

To verify that Apache Spark has been installed correctly, run the following command to start the Spark shell:

```
spark-shell
```

If the installation is successful, we should see the Spark shell prompt indicating that Spark is ready to use.

With these steps, we have successfully downloaded and installed Apache Spark on our Ubuntu system.

Configuration

Batch Processing

This code demonstrates how to use Apache Spark to read data from a Kafka topic named `market`, parse the JSON messages, and write the data to HDFS in JSON format. The process involves defining a schema for the Kafka messages, reading the streaming data from Kafka, parsing the JSON data, and writing the parsed data to HDFS with a specified trigger interval. The Spark session acts as the entry point to the Spark application, and the checkpointing mechanism ensures fault tolerance.

- **Schema Definition:** Defines the structure of the data read from Kafka.

```
schema = StructType([
    StructField("symbol", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("volume", IntegerType(), True),
    StructField("high", DoubleType(), True),
    StructField("low", DoubleType(), True),
    StructField("open", DoubleType(), True),
    StructField("timestamp", StringType(), True)
])
```

- **Reading Data from Kafka:** Reads the streaming data from a Kafka topic.

```
df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "market") \
    .option("failOnDataLoss", "false") \
    .load()
```

- **Parsing JSON Data:** Converts the JSON string data into a structured format.

```
df_parsed = df \
    .selectExpr("CAST(value AS STRING)") \
    .select(from_json(col("value").cast("string"), \
        schema).alias("data")) \
    .select("data.*")
```

- **Writing Data to HDFS:** Writes the parsed data to HDFS in JSON format with a trigger interval of 10 seconds.

```
query = df_parsed \
    .writeStream \
    .format("json") \
    .option("path", "hdfs://NameNode:9000/home/user/hadoop/\
        hdfs/namenode") \
    .option("checkpointLocation", "/tmp/checkpoint") \
    .outputMode("append") \
    .trigger(processingTime='10 seconds') \
    .start()
```

- **Awaiting Termination:** Keeps the streaming query active until it is manually stopped.

```
query.awaitTermination()
```

Streaming Processing

Below is an example of configuring Apache Spark for streaming data processing. This example reads data from a Kafka topic, processes it, and writes the results to a Kudu table.

- **Spark Session:** The entry point to programming Spark with the Dataset and DataFrame API.

```
spark = SparkSession.builder \
    .appName("KafkaToKudu") \
    .getOrCreate()
```

- **Schema Definition:** Defines the structure of the data read from Kafka.

```
schema = StructType([
    StructField("symbol", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("volume", IntegerType(), True),
    StructField("high", DoubleType(), True),
    StructField("low", DoubleType(), True),
    StructField("open", DoubleType(), True),
    StructField("timestamp", StringType(), True)
])
```

- **Reading from Kafka:** Reads the streaming data from a Kafka topic.

```
df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers",
"10.110.10.118:9092") \
    .option("subscribe", "market") \
    .option("failOnDataLoss", "false") \
    .load()
```

- **Parsing JSON Data:** Converts the JSON string data into a structured format.

```
df_parsed = df \
    .selectExpr("CAST(value AS STRING)") \
    .select(from_json(col("value").cast("string"),
schema).alias("data")) \
    .select("data.*")
```

- **Writing to Kudu:** A function to write each micro-batch of streaming data to Kudu.

```
def write_to_kudu(batch_df, batch_id):
    print(f"Batch ID: {batch_id}")
```

```

valid_data_df = batch_df.filter(col("symbol")
    .isNotNull())

valid_data_df.write \
    .format("org.apache.kudu.spark.kudu") \
    .option("kudu.master", "localhost:7051") \
    .option("kudu.table",
        "impala::default.StockData") \
    .mode("append") \
    .save()

print("Batch DataFrame:")
batch_df.show(truncate=False)

kudu_df = spark.read \
    .format("org.apache.kudu.spark.kudu") \
    .option("kudu.master", "localhost:7051") \
    .option("kudu.table",
        "impala::default.StockData") \
    .load()

print("Full Kudu Table:")
kudu_df.show(truncate=False)

```

- **Starting the Streaming Query:** Initiates the streaming query and writes the data to Kudu.

```

query = df_parsed \
    .writeStream \
    .foreachBatch(write_to_kudu) \
    .start()

query.awaitTermination()

```

This setup allows for real-time data processing using Apache Spark, reading from Kafka, and writing to Kudu.

6 Apache Kudu and Impala Integration

Describe the setup process for Apache Kudu and its integration with Apache Impala for real-time querying capabilities. Discuss the benefits of this integration and any special configuration steps.

6.1 Apache Kudu architecture and components

The core components of Kudu architecture include:

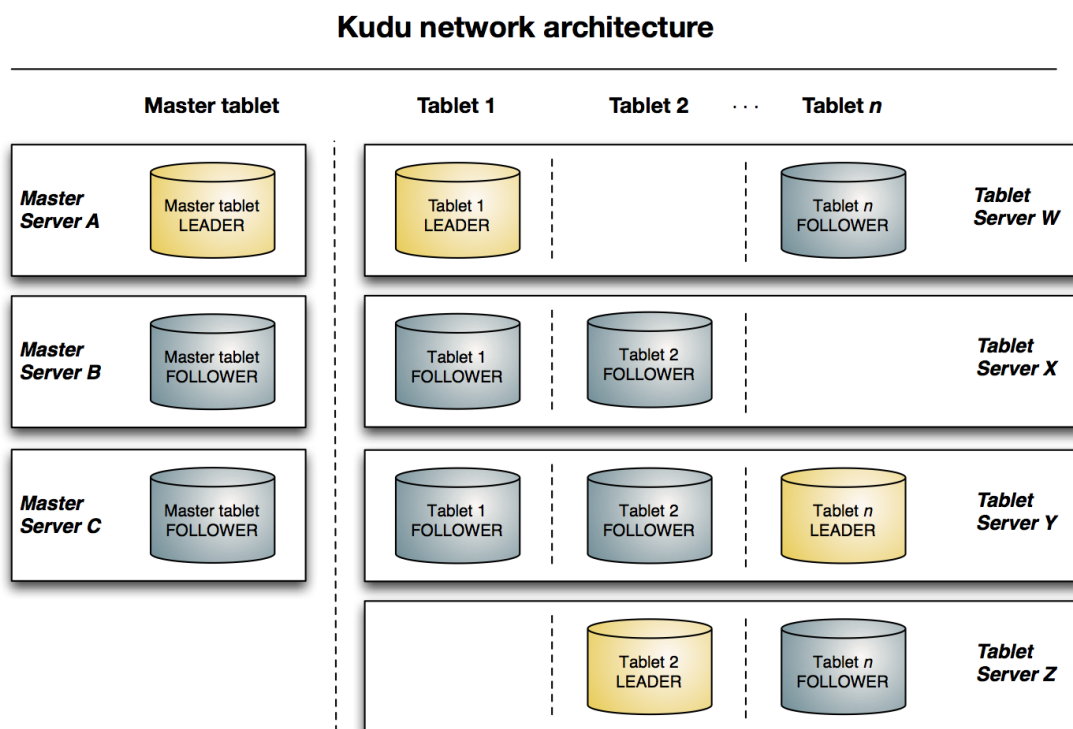


Figure 18. Apache Kudu Network Architecture [10]

- **Master Server:** Manages the overall metadata and tablet server assignments. It handles table creation, deletion, and schema changes, and keeps track of all the tablet servers.
- **Tablet Server:** Stores and serves tablets (Kudu's unit of data distribution). Each tablet server handles read and write requests for the tablets it hosts and performs data replication for fault tolerance.
- **Tablet:** A horizontal partition of a table, similar to a shard in other databases. Tablets are distributed across tablet servers.
- **Column Storage:** Kudu stores data in a columnar format, optimizing for high-throughput read and write operations. It supports efficient encoding and compression schemes to reduce storage footprint.

Understanding these components is crucial for configuring and optimizing Kudu for various use cases, particularly those involving high-speed analytics and frequent data modifications.

6.2 Apache Kudu Setup

This section outlines the steps required to install and set up Apache Kudu on an Ubuntu.

Prerequisites Installation

Begin by installing the necessary prerequisite libraries:

```
sudo apt-get install autoconf automake curl flex g++ gcc  
gdb git \ krb5-admin-server krb5-kdc krb5-user libkrb5-dev  
libsasl2-dev libsasl2-modules \ libsasl2-modules-gssapi-mit  
libssl-dev libtool lsb-release make ntp \ openjdk-8-jdk  
openssl patch pkg-config python rsync unzip vim-common
```

Optional: For support of Kudu's NVM (non-volatile memory) block cache, install the memkind library:

```
sudo apt-get install libmemkind0
```

If the provided memkind package is outdated, it should be built from source:

```
sudo apt-get install libnuma1 libnuma-dev  
git clone https://github.com/memkind/memkind.git  
cd memkind  
./build.sh --prefix=/usr  
sudo apt-get remove memkind  
sudo make install  
sudo ldconfig
```

Optional: To build documentation, install additional packages including ruby:

```
sudo apt-get install gem graphviz ruby-dev xsltproc zlib1g-dev
```

Optional: Install lsof if planning to run tests:

```
sudo apt-get install lsof
```

Kudu Installation

Clone the Kudu repository and navigate into the directory:

```
cmake
cmake --version
sudo snap install cmake --classic
git clone https://github.com/apache/kudu.git
cd kudu
mkdir build
cd build
cmake ..
make -j$(nproc)
export LD_LIBRARY_PATH=/opt/libs:$LD_LIBRARY_PATH
export PATH=/opt/tools/bin:$PATH
export LD_LIBRARY_PATH=/opt/libs:$LD_LIBRARY_PATH
source ~/.bashrc
mkdir /tmp/kudu-master
mkdir /tmp/kudu-tserver
```

To start kudu-master and kudu-tabletserver run these commands:

```
./bin/kudu-master --fs_wal_dir=/var/lib/kudu/master
--fs_data_dirs=/var/lib/kudu/master --webserver_doc_root=
/home/user/kudu/www --logtostderr

./bin/kudu-tserver --fs_wal_dir=/tmp/kudu-tserver
--fs_data_dirs=/tmp/kudu-tserver --tserver_master_addrs=
localhost:7051 --logtostderr
```

That was a comprehensive guide to setting up Apache Kudu on Ubuntu or Debian systems, detailing the installation of prerequisites, the Kudu software itself, and optional components based on specific needs.

6.3 Apache Impala Architecture and Components

The core components of Impala architecture include:

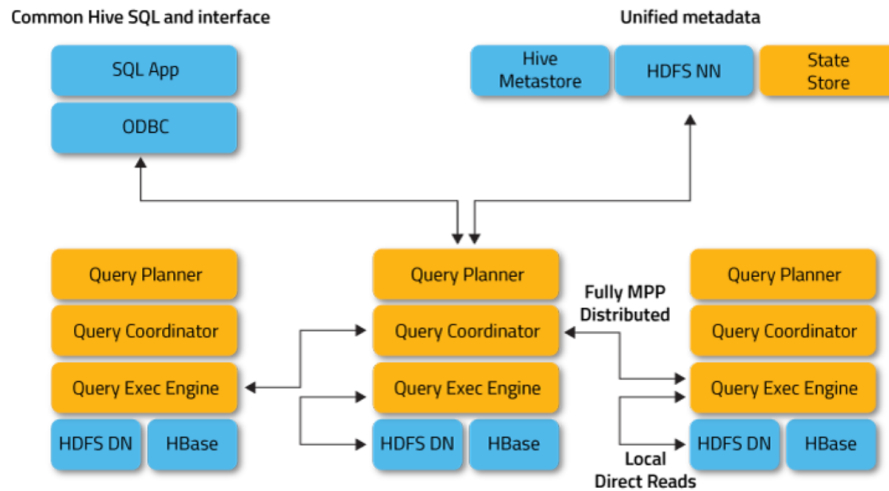


Figure 19. Apache Impala Architecture [20]

- **Impala Daemon (impalad):** The core component of Impala, responsible for executing queries. Each node in the cluster runs an instance of the Impala daemon, which manages query execution and data access on that node.
- **State Store (statestored):** Tracks the health and status of all Impala daemons in the cluster, providing vital information for query planning and execution.
- **Catalog Server (catalogd):** Manages metadata for Impala. It provides metadata caching and distribution, ensuring that all Impala daemons have up-to-date schema and table information.
- **Query Planner and Coordinator:** When a query is submitted, the planner converts it into a series of execution steps. The coordinator then distributes these steps to the appropriate Impala daemons for execution.
- **Distributed Query Execution:** Impala distributes query execution across multiple nodes in the cluster, leveraging data locality to optimize performance. Each node processes a portion of the data in parallel, and the results are aggregated and returned to the client.

Understanding these components is essential for configuring and optimizing Impala to achieve high performance and scalability for SQL-based analytics on big data.

6.4 Apache Impala Setup

Prerequisites

Before installing Impala we have to make sure that our system has the necessary dependencies:

```
sudo apt-get update
sudo apt-get install -y git cmake make g++ gcc libboost-all-dev
```

Additionally, Impala requires Java Development Kit (JDK) and Maven:

```
sudo apt-get install -y openjdk-8-jdk maven
```

Building Impala from Source

Clone the Impala repository from GitHub:

```
git clone https://github.com/apache/impala.git
cd impala
```

Build Impala using the provided scripts. This process will also build the necessary third-party dependencies:

```
source ./bin/impala-config.sh
./bin/bootstrap_development.sh
```

chose yes after getting this output:

```
This script will clobber some system settings. Are you sure
you want to continue? [yes/no]
```

Configuring Impala

After building, configure Impala by modifying configuration script in ‘.../impala/bin/impala-config.sh’.

Running Impala Services

Start the Impala services using the scripts found in the ‘bin’ directory:

```
bin/start-impala-cluster.sh
```

This script launches the Impala daemon, state store, and catalog services.

Verifying the Installation

Verify our Impala installation by connecting to the Impala shell and executing a query:

```
bin/impala-shell.sh
```

In the shell, try running:

```
SHOW DATABASES;
```

6.5 Apache Kudu with Apache Impala

Integrating Apache Kudu with Apache Impala allows users to perform SQL queries on Kudu tables using Impala. This combination offers an efficient way to execute real-time analytics on mutable, columnar data stored in Kudu. This section describes the necessary steps to configure both Apache Kudu and Apache Impala for seamless integration.

The architecture of the Kudu-Impala integration is illustrated the figure below:

Configuring Impala to Access Kudu

Impala needs to be aware of the Kudu Master servers to interact with Kudu. This is achieved by setting the `-kudu_master_hosts` flag in the Impala startup configuration, specifying the hostnames or IP addresses of the Kudu Master servers.

When running Impala and Kudu on a single machine for testing purposes, we can set this flag to `localhost`. In a production environment, list all Kudu Master server addresses, separated by commas.

This configuration enables Impala to communicate with the Kudu cluster.

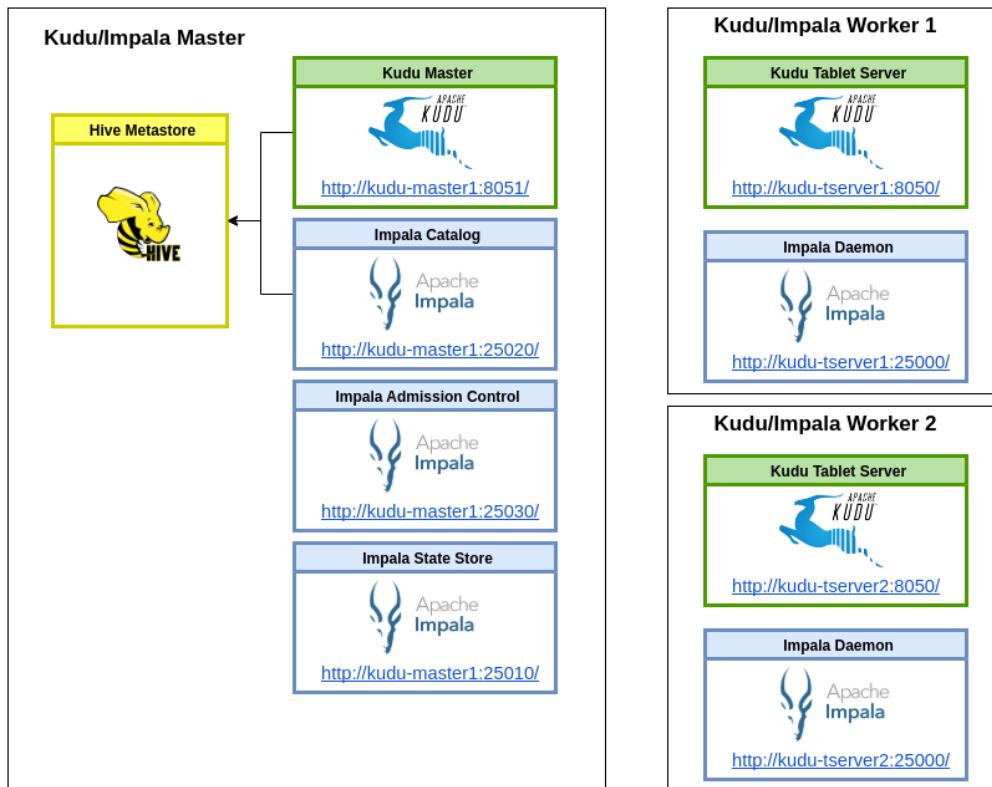


Figure 20. Kudu with Impala Architecture [21]

Creating Kudu Tables via Impala

Once Impala is configured to access Kudu, we can create Kudu tables directly through Impala SQL commands. Here's an example SQL statement to create a Kudu table:

```
CREATE TABLE kudu_table_name (
  id BIGINT,
  name STRING,
  PRIMARY KEY(id)
)
PARTITION BY HASH PARTITIONS 16
STORED AS KUDU;
```

This SQL command creates a Kudu table named 'kudu-table-name' with a primary key and a single string column. The table is partitioned into 16 hash partitions for efficient querying.

Querying Kudu Tables from Impala

With the Kudu table created, we can now perform SQL queries on this table directly from Impala. For example:

```
SELECT * FROM kudu_table_name WHERE id = 1;
```

This command retrieves rows from the ‘kudu-table-name’ table where the ‘id’ column equals 1.

Conclusion

Integrating Apache Kudu with Apache Impala combines Kudu’s efficient storage and fast updates with Impala’s powerful SQL querying capabilities. This integration enables real-time analytics on mutable datasets, making it an ideal solution for time-sensitive analytical workloads. By following the steps outlined in this section, we can set up a robust environment for real-time data processing and analytics.

6.6 Hadoop User Experience

Hue is an open-source web interface that simplifies the process of interacting with Hadoop and its ecosystem. It provides a user-friendly graphical interface for performing various tasks such as querying databases, managing files in HDFS, and monitoring job statuses [22].

Downloading and Installing Hue

To setup and run Hue, follow these steps:

```
git clone https://github.com/cloudera/hue.git
cd hue
make apps
build/env/bin/hue runserver
```

7 Web Interfaces for Monitoring and Management

Each of the discussed systems provides a web interface to facilitate monitoring, management, and interaction with the service. Accessing these interfaces allows users to view

metrics, manage configurations, and perform administrative tasks visually, enhancing the user experience.

Apache Hadoop Web Interfaces

Apache Hadoop offers two primary web interfaces for its components:

NameNode UI

The NameNode web interface can be accessed at http://namenode_host:9870, where is the hostname or IP address of the NameNode in our cluster. This interface displays information about the Hadoop filesystem, such as the overall health of the file system, blocks, and system capacity.

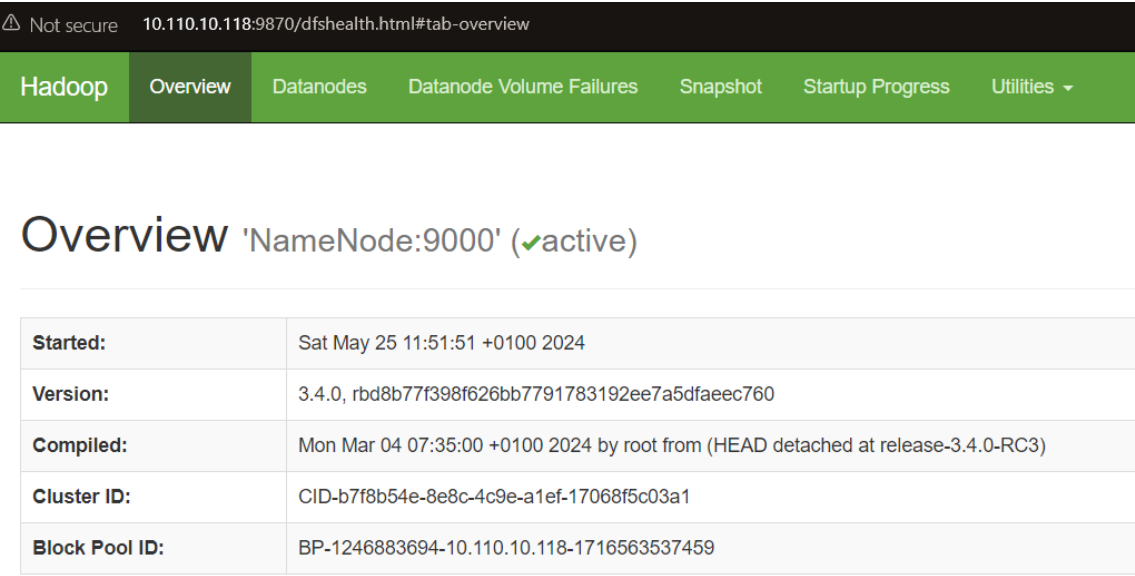


Figure 21. NameNode UI

YARN ResourceManager UI

The YARN ResourceManager web interface is available at http://resourcemanager_host:8088, providing insights into cluster resources, running applications, and node-level metrics.

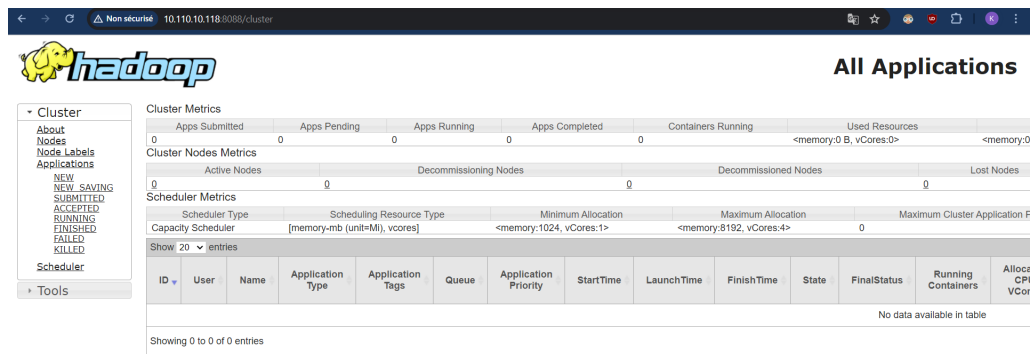


Figure 22. YARN UI

Apache Spark Web Interface

Apache Spark's web UI at http://your_spark_driver_host:4040 offers a comprehensive overview of Spark applications. It visualizes stages of execution, Spark tasks, Spark SQL queries, and provides JVM and executor metrics for debugging and optimization.

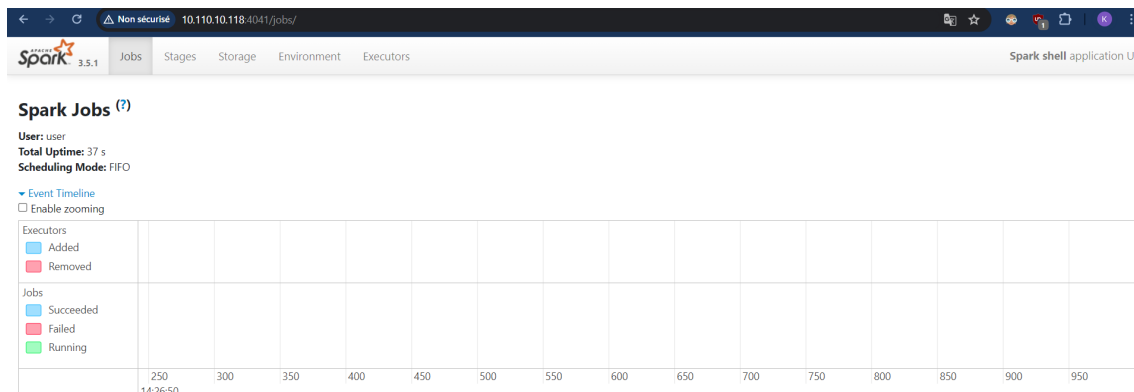


Figure 23. Spark UI

Apache Kudu Web Interface

Apache Kudu master servers and tablet servers each provide a web interface for monitoring and administration purposes. The Kudu Master UI is available at http://kudu_master_host:8051, and the Tablet Server UI is accessible at http://tablet_server_host:8050. They offer views into table schemas, tablet distributions, and server statistics.

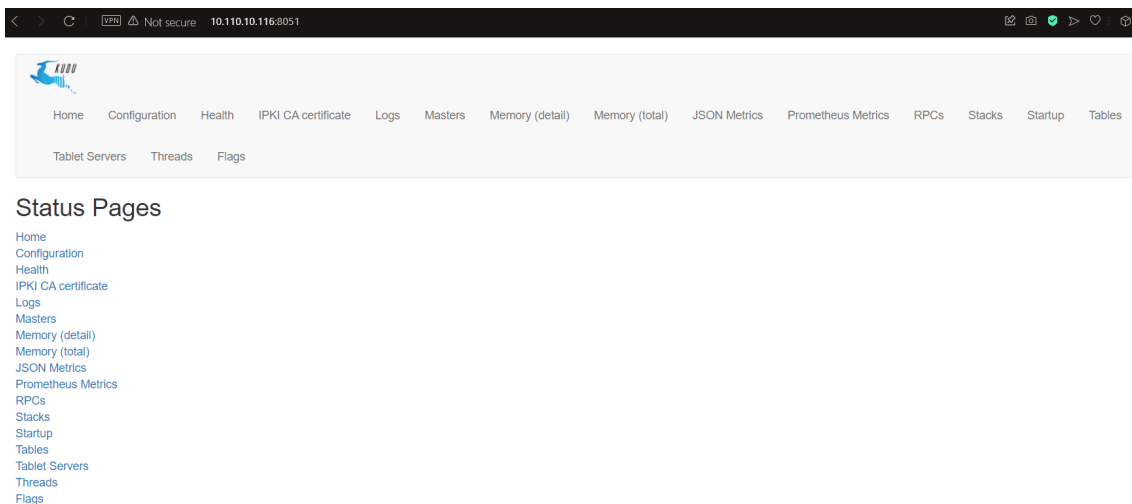


Figure 24. Kudu UI

Apache Impala Web Interface

For Apache Impala, the web UI is available at http://impalad_host:25000, giving insights into query plans, running queries, and system state. It's a crucial tool for analyzing the performance and efficiency of Impala queries.

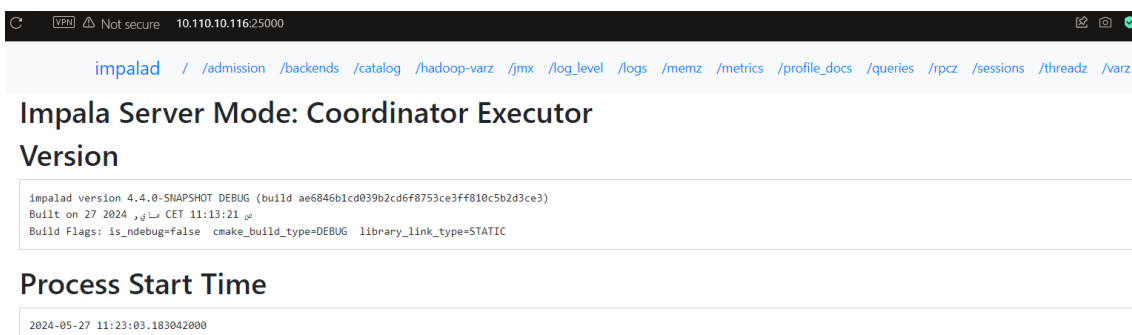


Figure 25. Impala UI

Hue web interface

Once Hue is installed and configured, you can access the Hue web interface through a web browser. By default, the Hue web interface is available at http://your_hue_server:8888. This interface provides a comprehensive suite of tools for managing and interacting with Hadoop and its ecosystem.

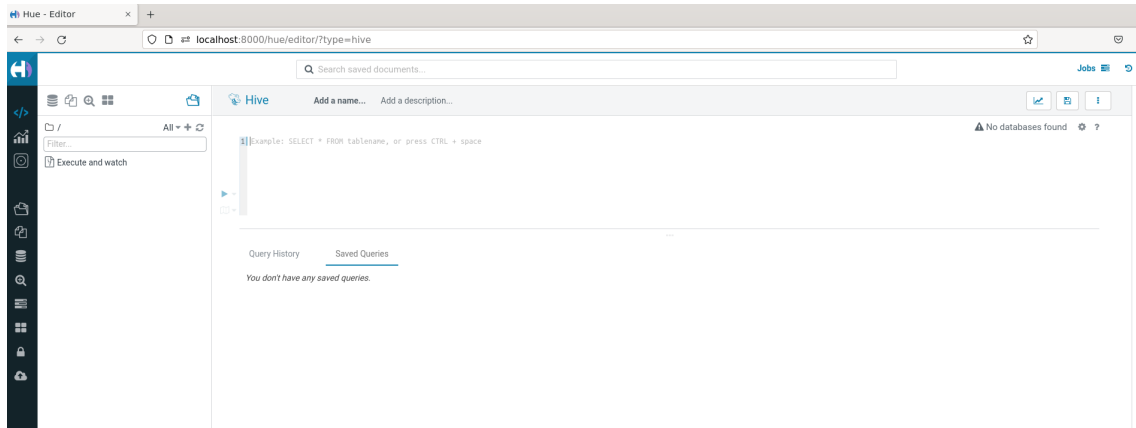


Figure 26. Hue UI

Chapter 4: Use case

1 Introduction

In this chapter, we present a practical use case to demonstrate the capabilities of our big data architecture. The chosen scenario involves the real-time simulation of stock market data. This use case highlights how our architecture can handle high-velocity data streams, perform real-time analytics, and ensure low-latency responses. The primary goal is to showcase the efficiency, scalability, and reliability of our system under realistic conditions.

2 Scenario Definition

The scenario involves simulating stock market data in real-time. We generate stock prices, trade volumes, and other relevant metrics for various stock symbols. This data is ingested, processed, stored, and queried using the components of our architecture:

- **Data Ingestion:** Apache Kafka is used to ingest the simulated stock market data.
- **Data Processing:** Apache Spark Streaming processes the ingested data in real-time.
- **Data Storage:** Processed data is stored in Apache Kudu for fast retrieval and updates.
- **Data Querying:** Apache Impala performs real-time queries on the processed data.

3 Demonstration

To set up the test environment, ensure the following components are installed and configured:

- Apache Kafka for data ingestion.
- Apache Hadoop for batch
- Apache Spark Streaming for real-time data processing.
- Apache Kudu for data storage.
- Apache Impala for querying the data.

3.1 Data Ingestion

We start by running Apache Kafka producer script to ingest the simulated stock market data using the following command:

```
python3 producer.py
```

The output of the Kafka producer script looks like this:

```
Sent data: {'symbol': 'AAPL', 'price': 266.36, 'volume': 2254, 'high': 292.73, 'low': 254.83, 'open': 274.34, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'GOOGL', 'price': 608.37, 'volume': 1686, 'high': 635.96, 'low': 595.1, 'open': 608.39, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'MSFT', 'price': 766.6, 'volume': 1571, 'high': 821.75, 'low': 697.06, 'open': 787.94, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'AMZN', 'price': 786.1, 'volume': 1013, 'high': 812.67, 'low': 718.32, 'open': 766.29, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'FB', 'price': 343.49, 'volume': 2211, 'high': 366.94, 'low': 336.0, 'open': 338.18, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'TSLA', 'price': 727.88, 'volume': 3917, 'high': 774.7, 'low': 699.74, 'open': 723.14, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'NFLX', 'price': 346.94, 'volume': 6551, 'high': 362.61, 'low': 331.86, 'open': 351.06, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'NVDA', 'price': 969.65, 'volume': 9935, 'high': 1059.9, 'low': 950.63, 'open': 1005.64, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'INTC', 'price': 906.7, 'volume': 2848, 'high': 994.51, 'low': 832.45, 'open': 931.6, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'AMD', 'price': 349.29, 'volume': 8507, 'high': 369.14, 'low': 325.99, 'open': 337.55, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'IBM', 'price': 570.81, 'volume': 9752, 'high': 597.62, 'low': 535.91, 'open': 585.13, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'CSCO', 'price': 881.59, 'volume': 1723, 'high': 957.69, 'low': 817.71, 'open': 898.55, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'ORCL', 'price': 812.78, 'volume': 6005, 'high': 861.06, 'low': 751.44, 'open': 858.41, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'QCOM', 'price': 685.64, 'volume': 1933, 'high': 743.99, 'low': 626.98, 'open': 672.88, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'PYPL', 'price': 991.02, 'volume': 7111, 'high': 1033.13, 'low': 936.57, 'open': 957.76, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'ADBE', 'price': 68.52, 'volume': 9850, 'high': 73.81, 'low': 66.86, 'open': 72.78, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'CRM', 'price': 821.42, 'volume': 3640, 'high': 867.2, 'low': 794.12, 'open': 796.66, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'SQ', 'price': 296.38, 'volume': 6528, 'high': 316.51, 'low': 286.31, 'open': 294.87, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'UBER', 'price': 278.7, 'volume': 4891, 'high': 296.24, 'low': 269.8, 'open': 292.97, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'LYFT', 'price': 803.17, 'volume': 8387, 'high': 881.48, 'low': 738.87, 'open': 753.07, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'BABA', 'price': 800.2, 'volume': 6791, 'high': 856.34, 'low': 784.48, 'open': 841.34, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'JD', 'price': 439.03, 'volume': 9651, 'high': 445.28, 'low': 420.39, 'open': 436.12, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'TSM', 'price': 774.82, 'volume': 1426, 'high': 819.72, 'low': 723.39, 'open': 808.45, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'SNE', 'price': 302.92, 'volume': 3621, 'high': 318.96, 'low': 283.45, 'open': 306.13, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'NIO', 'price': 755.62, 'volume': 7717, 'high': 786.46, 'low': 702.5, 'open': 726.91, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'RKT', 'price': 422.64, 'volume': 2477, 'high': 458.29, 'low': 391.24, 'open': 457.63, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'GM', 'price': 239.29, 'volume': 5498, 'high': 255.84, 'low': 224.11, 'open': 230.72, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'F', 'price': 654.41, 'volume': 6586, 'high': 692.69, 'low': 635.54, 'open': 644.97, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'GE', 'price': 187.98, 'volume': 3135, 'high': 201.07, 'low': 171.27, 'open': 195.69, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'BA', 'price': 438.61, 'volume': 9148, 'high': 474.62, 'low': 424.77, 'open': 452.78, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'PFE', 'price': 649.76, 'volume': 3870, 'high': 688.85, 'low': 640.48, 'open': 641.75, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'JNJ', 'price': 561.56, 'volume': 7539, 'high': 604.96, 'low': 550.77, 'open': 574.22, 'timestamp': '2024-05-29 15:09:24'}
Sent data: {'symbol': 'MRNA', 'price': 82.15, 'volume': 8115, 'high': 88.89, 'low': 75.66, 'open': 80.87, 'timestamp': '2024-05-29 15:09:24'}
```

Figure 27. Producer Output

3.2 Streaming

Creating empty kudu table

To create a Kudu table, first we start by running the impala-shell using the following command:

```
cd impala
./bin/impala-shell.sh
```

Then we create the stockdata table using the following code:

```
CREATE TABLE StockData
(
```

```

`symbol` STRING,
`timestamp` STRING,
price DOUBLE,
volume INT,
high DOUBLE,
low DOUBLE,
open DOUBLE,
PRIMARY KEY (`symbol`)
)
PARTITION BY HASH PARTITIONS 8
STORED AS KUDU;

```

The output of impala-shell after running the code is the following:

```

[localhost:21050] default> CREATE TABLE StockData
(
  `symbol` STRING,
  `timestamp` STRING,
  price DOUBLE,
  volume INT,
  high DOUBLE,
  low DOUBLE,
  open DOUBLE,
  PRIMARY KEY (`symbol`)
)
PARTITION BY HASH PARTITIONS 8
STORED AS KUDU;
Query: CREATE TABLE StockData
(
  `symbol` STRING,
  `timestamp` STRING,
  price DOUBLE,
  volume INT,
  high DOUBLE,
  low DOUBLE,
  open DOUBLE,
  PRIMARY KEY (`symbol`)
)
PARTITION BY HASH PARTITIONS 8
STORED AS KUDU
+-----+
| summary                |
+-----+
| Table has been created. |
+-----+
Fetched 1 row(s) in 4.53s
[localhost:21050] default>

```

Figure 28. Creating kudu table using impala-shell

Stream processing

To consume data from Kafka topic and pass it to Kudu we run the spark streaming job (page 40) using this command:

```
./bin/spark-submit --packages
org.apache.kudu:kudu-spark3_2.12:1.15.0,
org.apache.spark:spark-sql-kafka-0-10_2.12:3.2.0 market.py
```

The output result of running that job is the following:

```
e-2072919203-executor] Seeking to LATEST offset of partition market-0
24/05/29 15:52:43 INFO SubscriptionState: [Consumer clientId=consumer-spark-kafka-source-6b3c85a6-dab3-465a-b9a7-376b2b74fb8e-207291920
e-2072919203-executor] Resetting offset for partition market-0 to position FetchPosition{offset=357630, offsetEpoch=Optional.empty, cur
epoch=0}}.
24/05/29 15:52:43 INFO Executor: Finished task 0.0 in stage 20.0 (TID 37). 3067 bytes result sent to driver
24/05/29 15:52:43 INFO TaskSetManager: Finished task 0.0 in stage 20.0 (TID 37) in 516 ms on 10.110.10.116 (executor driver) (1/1)
24/05/29 15:52:43 INFO TaskSchedulerImpl: Removed TaskSet 20.0, whose tasks have all completed, from pool
24/05/29 15:52:43 INFO DAGScheduler: ResultStage 20 (start at NativeMethodAccessorImpl.java:0) finished in 0,523 s
24/05/29 15:52:43 INFO DAGScheduler: Job 21 is finished. Cancelling potential speculative or zombie tasks for this job
24/05/29 15:52:43 INFO TaskSchedulerImpl: Killing all running tasks in stage 20: Stage finished
24/05/29 15:52:43 INFO DAGScheduler: Job 21 finished: start at NativeMethodAccessorImpl.java:0, took 0,524537 s
+-----+-----+-----+-----+-----+-----+
|symbol|price|volume|high|low|open|timestamp|
+-----+-----+-----+-----+-----+-----+
|AAPL|674.51|5201|731.57|664.46|676.83|2024-05-29 15:52:38|
|GOOGL|80.13|1431|83.74|73.27|75.21|2024-05-29 15:52:38|
|MSFT|111.31|2968|115.12|102.43|110.49|2024-05-29 15:52:38|
|AMZN|662.76|9103|726.15|625.38|686.03|2024-05-29 15:52:38|
|FB|660.32|4379|675.04|604.69|644.64|2024-05-29 15:52:38|
|TSLA|573.19|4596|617.55|536.2|596.39|2024-05-29 15:52:38|
|NFLX|991.37|3419|1026.62|929.11|1007.45|2024-05-29 15:52:38|
|NVDA|553.63|2331|566.72|522.86|531.6|2024-05-29 15:52:38|
|INTC|853.94|3989|904.24|828.97|879.07|2024-05-29 15:52:38|
|AMD|250.78|3994|268.53|228.0|248.56|2024-05-29 15:52:38|
|IBM|143.68|5031|150.29|130.44|133.13|2024-05-29 15:52:38|
|CSCO|364.4|2129|386.73|338.37|365.34|2024-05-29 15:52:38|
|ORCL|533.83|6902|582.09|506.16|507.24|2024-05-29 15:52:38|
|QCOM|367.82|6109|375.23|333.06|347.09|2024-05-29 15:52:38|
|PYPL|499.67|4477|529.84|455.69|457.29|2024-05-29 15:52:38|
|ADBE|906.42|6152|931.29|825.48|905.77|2024-05-29 15:52:38|
|CRM|691.04|1385|741.79|679.46|710.69|2024-05-29 15:52:38|
|SQ|741.58|3031|792.86|708.24|715.4|2024-05-29 15:52:38|
|UBER|696.12|6427|722.47|641.28|653.75|2024-05-29 15:52:38|
|LYFT|360.35|3345|381.4|352.02|371.21|2024-05-29 15:52:38|
+-----+-----+-----+-----+-----+-----+
only showing top 20 rows

Full Kudu Table:
24/05/29 15:52:43 INFO SparkContext: Starting job: start at NativeMethodAccessorImpl.java:0
24/05/29 15:52:43 INFO DAGScheduler: Got job 22 (start at NativeMethodAccessorImpl.java:0) with 1 output partitions
24/05/29 15:52:43 INFO DAGScheduler: Final stage: ResultStage 21 (start at NativeMethodAccessorImpl.java:0)
24/05/29 15:52:43 INFO DAGScheduler: Parents of final stage: List()
24/05/29 15:52:43 INFO DAGScheduler: Missing parents: List()
24/05/29 15:52:43 INFO DAGScheduler: Submitting ResultStage 21 (MapPartitionsRDD[83] at start at NativeMethodAccessorImpl.java:0), whic
24/05/29 15:52:43 INFO MemoryStore: Block broadcast_21 stored as values in memory (estimated size 32.1 KiB, free 434.1 MiB)
24/05/29 15:52:43 INFO MemoryStore: Block broadcast_21_piece0 stored as bytes in memory (estimated size 13.1 KiB, free 434.1 MiB)
24/05/29 15:52:43 INFO BlockManagerInfo: Added broadcast_21_piece0 in memory on 10.110.10.116:37007 (size: 13.1 KiB, free: 434.3 MiB)
```

Figure 29. Spark Streaming job terminal

This is the output on the Spark UI:

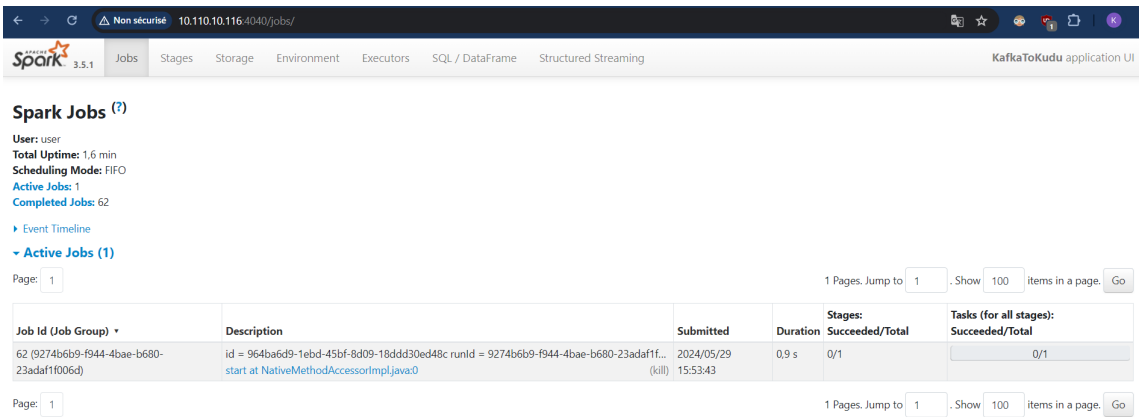


Figure 30. Spark Active Job

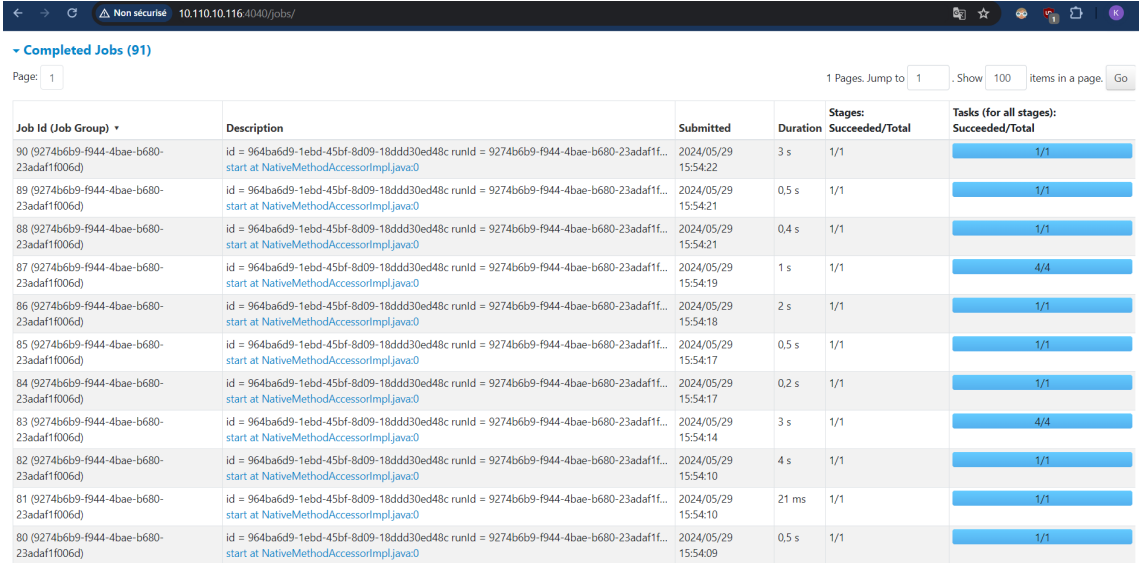


Figure 31. Spark Completed Jobs

Querying Kudu table using Impala

As an example of querying the Kudu table using Impala we run the query in Impala-shell

```
SELECT * FROM stockdata ;
```

The result of this query:

```
[localhost:21050] default> select * from stockdata ;
Query: select * from stockdata
Query submitted at: 2024-05-29 15:43:23 (Coordinator: http://impala:25000)
Query state can be monitored at: http://impala:25000/query_plan?query_id=0d44d1c9912568a4:36bbb63100000000
```

symbol	timestamp	price	volume	high	low	open
ADBE	2024-05-27 16:24:06	163.98	1372	179.84	161.36	174.42
CRM	2024-05-27 16:24:06	626.57	4984	659.22	616.89	633.53
JD	2024-05-27 16:24:06	421.3	7029	441.66	382.61	432.12
LOW	2024-05-27 16:24:06	150.16	2698	155.04	145.84	150.95
MRNA	2024-05-27 16:24:06	535.79	9215	557.92	506.79	544.57
NFLX	2024-05-27 16:24:06	809.22	1401	885.1	750.23	762.2
RKT	2024-05-27 16:24:06	883.61	8002	934.7	824.17	843.71
SNE	2024-05-27 16:24:06	251.45	3177	256.51	240.86	254.24
UBER	2024-05-27 16:24:06	564.96	6399	585.57	538.08	539.08
WBA	2024-05-27 16:24:06	398.11	7360	409.68	371.81	389.16
WMT	2024-05-27 16:24:06	789.36	4958	867.96	722.79	841.43
MNST	2024-05-27 16:24:06	465.93	1266	481.92	429.61	475.73
MRK	2024-05-27 16:24:06	374.82	6078	398.89	348.93	383.24
MSFT	2024-05-27 16:24:06	255.66	6887	270.77	233.15	243.51
NIO	2024-05-27 16:24:06	361.32	7829	370.07	351.13	361.27
AAPL	2024-05-27 16:24:06	370.71	6022	401.68	365.69	380.92
AMD	2024-05-27 16:24:06	318.27	5960	331.0	306.2	309.45
QCOM	2024-05-27 16:24:06	225.5	6686	240.33	221.97	238.07
DPZ	2024-05-27 16:24:06	850.74	9634	896.65	828.25	857.41
GILD	2024-05-27 16:24:06	152.82	9013	163.85	148.86	152.69
GOOGL	2024-05-27 16:24:06	117.52	7262	124.6	106.7	113.41
HD	2024-05-27 16:24:06	492.2	7986	516.92	484.22	489.57
BMJ	2024-05-27 16:24:06	579.0	1722	602.9	544.26	581.5
F	2024-05-27 16:24:06	808.02	5222	824.2	781.71	810.83
GM	2024-05-27 16:24:06	326.88	1482	335.42	309.53	313.42
MCD	2024-05-27 16:24:06	580.17	8601	620.55	528.77	541.5
ORCL	2024-05-27 16:24:06	705.64	1262	761.32	642.09	758.54
PFE	2024-05-27 16:24:06	193.52	5677	206.06	175.31	203.14
BABA	2024-05-27 16:24:06	192.73	8617	201.51	176.47	190.49
BNTX	2024-05-27 16:24:06	319.97	5073	343.73	295.22	308.45
GE	2024-05-27 16:24:06	92.22	8552	96.22	91.21	95.22
IBM	2024-05-27 16:24:06	159.16	7739	173.82	154.31	162.68
JNJ	2024-05-27 16:24:06	310.13	7176	326.37	289.7	303.79
LYFT	2024-05-27 16:24:06	350.96	1121	357.8	317.15	349.89
NVDA	2024-05-27 16:24:06	678.39	8087	699.15	620.26	639.72
BA	2024-05-27 16:24:06	855.67	5576	931.9	838.47	866.82
CSCO	2024-05-27 16:24:06	850.99	2289	872.76	811.52	833.25
FB	2024-05-27 16:24:06	816.06	4553	868.06	804.63	840.65
KO	2024-05-27 16:24:06	363.38	4446	373.84	344.75	364.38
PEP	2024-05-27 16:24:06	211.07	6616	224.14	195.1	207.68
PYPL	2024-05-27 16:24:06	818.4	6008	864.87	773.29	852.14
TGT	2024-05-27 16:24:06	316.2	7380	332.24	284.72	298.99
YUM	2024-05-27 16:24:06	164.81	5364	174.29	163.13	167.92
ABT	2024-05-27 16:24:06	931.7	7498	983.82	850.86	878.04
AMZN	2024-05-27 16:24:06	301.89	8882	310.08	294.6	308.17
INTC	2024-05-27 16:24:06	877.65	1418	925.73	819.53	853.87
SBUX	2024-05-27 16:24:06	659.84	8500	723.04	623.48	708.06
SQ	2024-05-27 16:24:06	764.58	8045	831.8	706.63	790.29
TSLA	2024-05-27 16:24:06	393.51	2647	408.03	360.4	379.8
TSM	2024-05-27 16:24:06	287.77	1705	312.92	266.13	303.66

Fetches 50 row(s) in 0.12s

Figure 32. Select Query

To see more details on the Impala Query we can check the Impala UI :

Queries

This page lists all running queries, completed queries that are archived in memory, and imported query profiles.

The in-memory archive size is controlled by `--query_log_size` and `--query_log_size_in_bytes` command line parameters. The length of the statements are controlled by `--query_stat_size` command line parameter.

0 queries in flight

Query ID	Action	User	Default Db	Query Type	Start Time	Duration	Mem Usage	Scan Progress	Bytes Read	State	Last Event	# rows fetched	Resource Pool	Statement
						Queued Duration	Mem Estimate	Query Progress	Bytes Sent					

0 waiting to be closed [\[?\]](#)

Query ID	Action	User	Default Db	Query Type	Start Time	Waiting Time	Duration	Mem Usage	Scan Progress	Bytes Read	State	Last Event	# rows fetched	Resource Pool	Statement
							Queued Duration	Mem Estimate	Query Progress	Bytes Sent					

Last 200 Completed Queries

Query ID	User	Default Db	Query Type	Start Time	End Time	Duration	Mem Usage	Scan Progress	Bytes Read	State	# rows fetched	Resource Pool	Statement
						Queued Duration	Mem Estimate	Query Progress	Bytes Sent				
4b41fa597bf1193d60c09fa800000000	user	default	QUERY	2024-05-29 15:38:13.814	2024-05-29 15:38:14.024	210.847ms 0	5.17 MB 62.98 MB	8 / 8 (100%) 4 / 4 (100%)	3.67 KB 3.51 KB	FINISHED	50	default-pool	select * from stockdata
d0447ae1c9449eb37ca0437b000000000	user	default	DDL	2024-05-29 15:49:38.055	2024-05-29 15:49:42.583	4s528ms 0	0 0	N/A N/A	0 0	FINISHED	1		CREATE TABLE StockData (`symbol` STRING, `timestamp` STRING, price DOUBLE, volume INT, high DOUBLE, low DOUBLE, open DOUBLE, PRIMARY KEY (`symbol`,`timestamp`)) PARTITION BY HASH PARTITIONS 8 STORED AS KUDU

Figure 33. Impala UI Queries

Using Hue browser

By using the Hue browser, we can write, run, and manage Impala queries in a user-friendly interface.

This is how the Hue interface looks like:

localhost:8000/hue/editor?editor=75

Search saved documents...

Impala

SELECT * FROM stockdata

Query 444f202674b269ad:a58f865508000000: 62% Complete (5 out of 8)
Query 444f202674b269ad:a58f865508000000: 100% Complete (8 out of 8)
Query 444f202674b269ad:a58f865508000000: 100% Complete (8 out of 8)

Query History Saved Queries Results (50)

	symbol	timestamp	price	volume	high	low	open
1	MNST	2024-05-27 15:25:03	509.64	2506	533.29	475.95	511.97
2	MRK	2024-05-27 15:25:03	689.05	9613	701.96	632.69	655.29
3	MSFT	2024-05-27 15:25:03	994.85	2270	1055.4	901.15	995.41
4	NIO	2024-05-27 15:25:03	772.61	7456	840.39	700.08	778.55
5	ADBE	2024-05-27 15:25:03	916.31	4450	935.64	870.49	928.38
6	CRM	2024-05-27 15:25:03	508.79	9168	543.11	501.89	505.15
7	JD	2024-05-27 15:25:03	731.92	9682	789.29	684.38	739.1
8	LOW	2024-05-27 15:25:03	687.47	9263	722.07	662.96	668.94
9	MRNA	2024-05-27 15:25:03	990.98	3066	1014.09	928.49	981.48
10	NFLX	2024-05-27 15:25:03	936.26	2159	964.42	924.96	938.64
11	RKT	2024-05-27 15:25:03	652.51	9325	675.57	630.85	643.44

Figure 34. Hue Browser Table Result

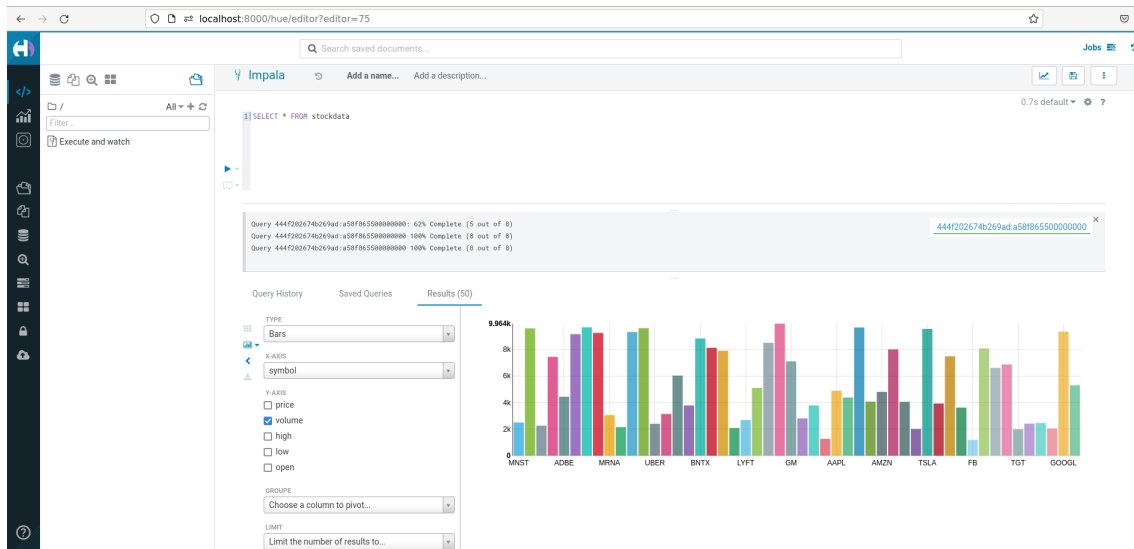


Figure 35. Hue browser Graph Result

3.3 Batch

Batch data storage

To consume data from Kafka topic and pass it to HDFS we run the HDFS Sink connector (page 35) we should get the following output: we can Check the NameNode UI to check

```

startup, the HDFS Connector restores offsets from filenames in HDFS. In the absence of files in HDFS, the connector will attempt to find offsets for its consumer group in the 'consumer_offsets' topic. If offsets are not found, the consumer will rely on the reset policy specified in the 'consumer.auto.offset.reset' property to start exporting data to HDFS. (io.confluent.connect.hdfs3.Hdfs3SinkTask:94)
[2024-05-29 16:08:33.020] INFO [hdfs3-sink-connector|task-0] WorkersSinkTask[id=hdfs3-sink-connector-0] Sink task finished initialization and start (org.apache.kafka.connect.runtime.WorkersSinkTask:326)
[2024-05-29 16:08:33.021] INFO [hdfs3-sink-connector|task-0] Executing sink task (org.apache.kafka.connect.runtime.WorkersSinkTask:207)
[2024-05-29 16:08:33.045] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Cluster ID: whx_e66b05-01ad7Mthw (org.apache.kafka.clients.Metadata:292)
[2024-05-29 16:08:33.046] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Discovered group coordinator NameNode:9092 (id: 2 47483647 rack: null) (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:908)
[2024-05-29 16:08:33.049] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] (Re-)joining group (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:576)
[2024-05-29 16:08:33.064] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Request joining group due to: need to re-join with the given member-id: connector-consumer-hdfs3-sink-connector-0-46424d58-ccc7-47da-a260-0b2a8f42d1da (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:1072)
[2024-05-29 16:08:33.065] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Request joining group due to: rebalance failed due to 'The group member needs to have a valid member id before actually entering a consumer group.' (MemberIdRequiredException) (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:1072)
[2024-05-29 16:08:33.065] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] (Re-)joining group (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:576)
[2024-05-29 16:08:33.067] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Successfully joined group with generation GenerationId=9, memberId=connector-consumer-hdfs3-sink-connector-0-46424d58-ccc7-47da-a260-0b2a8f42d1da, protocol=range (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:637)
[2024-05-29 16:08:33.073] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Finished assignment for group at generation 9: (connector-consumer-hdfs3-sink-connector-0-46424d58-ccc7-47da-a260-0b2a8f42d1da=Assignment(partitions=[market-0]) (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:718)
[2024-05-29 16:08:33.078] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Successfully synced group in generation GenerationId=9, memberId=connector-consumer-hdfs3-sink-connector-0-46424d58-ccc7-47da-a260-0b2a8f42d1da, protocol=range (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:812)
[2024-05-29 16:08:33.079] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Notifying assignor about the new Assignment(partitions=[market-0]) (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:313)
[2024-05-29 16:08:33.081] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Adding newly assigned partitions: market-0 (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:325)
[2024-05-29 16:08:33.090] INFO [hdfs3-sink-connector|task-0] Setting offset for partition market-0 to the committed offset FetchPosition[offset=224000, offsetEpoch=Optional.empty, currentLeader=LeaderAndEpoch(leader=Optional(NameNode:9092 (id: 0 rack: null)), epoch=0)] (org.apache.kafka.clients.consumer.internals.ConsumerCoordinator:196)
[2024-05-29 16:08:33.097] INFO [hdfs3-sink-connector|task-0] Started recovery for topic partition market-0 (io.confluent.connect.hdfs3.TopicPartitionWriter:269)
[2024-05-29 16:08:33.223] INFO [hdfs3-sink-connector|task-0] Successfully acquired lease for hdfs://NameNode:9090/logs/market/0/log (io.confluent.connect.hdfs3.wal.FSMA:67)
[2024-05-29 16:08:33.469] INFO [hdfs3-sink-connector|task-0] Finished recovery for topic partition market-0 (io.confluent.connect.hdfs3.TopicPartitionWriter:288)
[2024-05-29 16:08:33.469] INFO [hdfs3-sink-connector|task-0] [Consumer clientId=connector-consumer-hdfs3-sink-connector-0, groupId=connect-hdfs3-sink-connector] Seeking to offset 224000 for partition market-0 (org.apache.kafka.clients.consumer.KafkaConsumer:1502)
[2024-05-29 16:08:33.657] INFO [hdfs3-sink-connector|task-0] committing files after waiting for rotateIntervalMs time but less than flush.size records available. (io.confluent.connect.hdfs3.TopicPartitionWriter:448)
[2024-05-29 16:08:33.678] INFO [hdfs3-sink-connector|task-0] Successfully acquired lease for hdfs://NameNode:9090/logs/market/0/log (io.confluent.connect.hdfs3.wal.FSMA:67)
[2024-05-29 16:08:33.718] INFO [hdfs3-sink-connector|task-0] Committed hdfs://NameNode:9090/topics/market/partition=0/market-0+0000224000-0000224999.json for market-0 (io.confluent.connect.hdfs3.TopicPartitionWriter:822)
[2024-05-29 16:08:33.751] INFO [hdfs3-sink-connector|task-0] committing files after waiting for rotateIntervalMs time but less than flush.size records available. (io.confluent.connect.hdfs3.TopicPartitionWriter:448)
[2024-05-29 16:08:34.183] INFO [hdfs3-sink-connector|task-0] Committed hdfs://NameNode:9090/topics/market/partition=0/market-0+0000225000-0000225999.json for market-0 (io.confluent.connect.hdfs3.TopicPartitionWriter:822)
[2024-05-29 16:08:34.208] INFO [hdfs3-sink-connector|task-0] committing files after waiting for rotateIntervalMs time but less than flush.size records available. (io.confluent.connect.hdfs3.TopicPartitionWriter:448)
[2024-05-29 16:08:34.236] INFO [hdfs3-sink-connector|task-0] Committed hdfs://NameNode:9090/topics/market/partition=0/market-0+0000226000-0000226999.json for market-0 (io.confluent.connect.hdfs3.TopicPartitionWriter:822)
[2024-05-29 16:08:34.249] INFO [hdfs3-sink-connector|task-0] committing files after waiting for rotateIntervalMs time but less than flush.size records available. (io.confluent.connect.hdfs3.TopicPartitionWriter:448)

```

Figure 36. HDFS Connector output

if the data and number of blocks are increasing to ensure that HDFS is getting the data successfully:

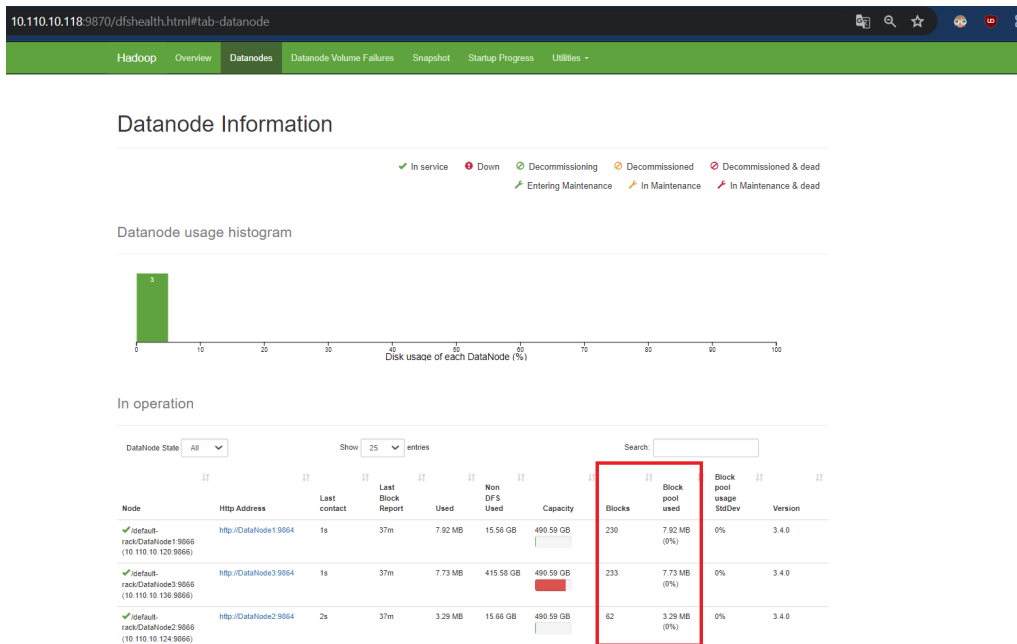


Figure 37. HDFS number of blocks

Spark batch processing

Or instead of saving data directly to HDFS we can pass it through Spark to do some processing before saving it for that we can run the Spark job that consumes data from kafka topic and saves it to HDFS, this is its output on terminal:

```
24/05/29 16:12:50 INFO FileFormatWriter: Start to commit write Job e3bf0d5f-381a-4ed5-ae05-9c2aefca87f.
24/05/29 16:12:50 INFO CheckpointFileManager: Writing atomically to hdfs://NameNode:9000/home/user/hadoop/hdfs/namenode/_spark_metadata/4.d75a376f-473f-4ca1-a4bc-125c6d3dc222.tmp
24/05/29 16:12:50 INFO CheckpointFileManager: Renamed temp file hdfs://NameNode:9000/home/user/hadoop/hdfs/namenode/_spark_metadata/4.d75a376f-473f-4ca1-a4bc-125c6d3dc222.tmp to hdfs://NameNode:9000/home/user/hadoop/hdfs/namenode/_spark_metadata/4
24/05/29 16:12:50 INFO NonFileWriteCommitProtocol: Committed batch 4
24/05/29 16:12:50 INFO FileFormatWriter: Write Job e3bf0d5f-381a-4ed5-ae05-9c2aefca87f committed. Elapsed time: 22 ms.
24/05/29 16:12:50 INFO FileFormatWriter: Finished processing stats for write job e3bf0d5f-381a-4ed5-ae05-9c2aefca87f.
24/05/29 16:12:50 INFO CheckpointFileManager: Writing atomically to hdfs://NameNode:9000/tmp/checkpoint/commits/4 using temp file hdfs://NameNode:9000/tmp/checkpoint/commits/4.5b552662-6610-40bb-a64c-37526780ce9.tmp
24/05/29 16:12:50 INFO CheckpointFileManager: Renamed temp file hdfs://NameNode:9000/tmp/checkpoint/commits/4.5b552662-6610-40bb-a64c-37526780ce9.tmp to hdfs://NameNode:9000/tmp/checkpoint/commits/4
24/05/29 16:12:50 INFO MicroBatchExecution: Streaming query made progress: {
  "id" : "1908014-7377-4721-894a-c7a705090975",
  "runId" : "1ec0c16-800b-4c09-8306-90c25adff93",
  "name" : null,
  "timestamp" : "2024-05-29T15:12:50.000Z",
  "batchId" : 4,
  "numInputRows" : 400,
  "inputRowsPerSecond" : 43.30410306376529,
  "processedRowsPerSecond" : 422.3864836325238,
  "durations" : {
    "addBatch" : 459,
    "commitOffsets" : 422,
    "getBatch" : 0,
    "latestOffset" : 5,
    "queryPlanning" : 20,
    "triggerExecution" : 947,
    "waitCommit" : 26
  },
  "stateOperators" : [ ],
  "sources" : [ {
    "description" : "KafkaV2[Subscribe[market]]",
    "startOffset" : {
      "market" : {
        "0" : 417030
      }
    },
    "endOffset" : {
      "market" : {
        "0" : 417430
      }
    },
    "latestOffset" : null,
    "numInputRows" : 400,
    "inputRowsPerSecond" : 43.30410306376529,
    "processedRowsPerSecond" : 422.3864836325238
  } ],
  "sink" : {
    "description" : "FileSink[hdfs://NameNode:9000/home/user/hadoop/hdfs/namenode]",
    "numOutputRows" : -1
  }
}
```

Figure 38. Spark Batch Output

Querying (SparkSQL)

For querying Batch data stored in HDFS we use SparkSQL, the following are the results of some examples of queries executed using SparkSQL:

```
24/05/31 14:02:08 INFO Executor: Finished task 0.0 in stage 3.0 (TID 8). 5859 bytes result sent to driver
24/05/31 14:02:08 INFO TaskSetManager: Finished task 0.0 in stage 3.0 (TID 8) in 218 ms on NameNode (executor driver) (1/1)
24/05/31 14:02:08 INFO TaskSchedulerImpl: Removed TaskSet 3.0, whose tasks have all completed, from pool
24/05/31 14:02:08 INFO DAGScheduler: ResultStage 3 (showString at NativeMethodAccessorImpl.java:0) finished in 0,239 s
24/05/31 14:02:08 INFO DAGScheduler: Job 2 is finished. Cancelling potential speculative or zombie tasks for this job
24/05/31 14:02:08 INFO TaskSchedulerImpl: Killing all running tasks in stage 3: Stage finished
24/05/31 14:02:08 INFO DAGScheduler: Job 2 finished: showString at NativeMethodAccessorImpl.java:0, took 0,257023 s
24/05/31 14:02:08 INFO CodeGenerator: Code generated in 9.646218 ms
```

symbol	max_price	max_volume	max_high	max_low	max_open	max_timestamp
AAPL	999.04	9950	1063.05	948.38	1004.17	2024-05-31 13:55:59
ABT	999.25	9954	1048.98	981.45	1017.41	2024-05-31 13:55:59
ADBE	988.49	9967	1040.75	945.66	989.51	2024-05-31 13:55:59
AMD	997.66	9944	1090.25	943.93	1030.39	2024-05-31 13:55:59
AMZN	996.67	9847	1066.19	966.89	1009.78	2024-05-31 13:55:59
BA	998.58	9926	1068.26	979.88	1006.86	2024-05-31 13:55:59
BABA	995.76	9904	1071.41	978.75	1028.28	2024-05-31 13:55:59
BMY	994.1	9953	1076.93	945.78	1040.53	2024-05-31 13:55:59
BNTX	999.63	10000	1060.76	987.74	1045.01	2024-05-31 13:55:59
CRM	999.64	9978	1068.18	951.19	1031.14	2024-05-31 13:55:59
CSCO	994.74	9895	1083.74	963.82	1061.66	2024-05-31 13:55:59
DPZ	997.57	9948	1082.05	955.41	1034.7	2024-05-31 13:55:59
F	996.88	9990	1083.66	962.51	1049.44	2024-05-31 13:55:59
FB	998.26	9997	1072.51	985.87	1010.19	2024-05-31 13:55:59
GE	990.45	9961	1077.67	954.05	1036.07	2024-05-31 13:55:59
GILD	997.52	9894	1086.18	980.75	1075.49	2024-05-31 13:55:59
GM	995.02	9995	1075.51	960.11	1016.19	2024-05-31 13:55:59
GOOGL	999.47	9970	1095.39	965.08	1024.88	2024-05-31 13:55:59
HD	993.24	9926	1077.48	959.21	1067.77	2024-05-31 13:55:59
IBM	997.48	9908	1083.55	962.25	1060.91	2024-05-31 13:55:59

only showing top 20 rows

```
24/05/31 14:02:08 INFO FileSourceStrategy: Pushed Filters:
```

Figure 39. Querying with Spark

4 Conclusion

The real-time stock market data simulation demonstrates the robustness and efficiency of our big data architecture. By handling high-velocity data streams and providing low-latency responses, our system proves to be highly capable for real-time analytics. The performance metrics obtained from the tests validate the scalability and reliability of our architecture, making it suitable for various real-world applications.

Conclusion

In conclusion, the implementation of a big data architecture for native analytics on fast and changing data, as demonstrated in this project, underscores the critical role of modern data processing technologies like Apache Kafka, Apache Hadoop, Apache Spark, Apache Kudu, and Apache Impala. These technologies collectively provide a robust, scalable, and efficient solution for real-time data ingestion, processing, storage, and querying.

The project successfully addressed the challenges posed by traditional data architectures, which struggle to keep pace with the exponential growth in data volume, velocity, and variety. By leveraging the strengths of each component, the proposed architecture ensures low-latency responses and high-throughput processing, making it ideal for applications that require immediate insights and agile decision-making.

The practical use case of simulating stock market data highlighted the architecture's capability to handle high-velocity data streams and perform real-time analytics effectively. The integration of Apache Kudu with Apache Impala proved particularly beneficial, enabling seamless real-time querying on mutable datasets and ensuring that the system can adapt to rapidly changing data.

Furthermore, the project's emphasis on future-proofing the architecture ensures that it remains adaptable to emerging technologies and evolving data processing requirements. This forward-thinking approach positions the architecture as a versatile and enduring solution for a wide range of industries facing the complexities of big data.

Overall, this project not only demonstrates the feasibility and advantages of modern big data architectures but also provides a comprehensive guide for deploying and configuring such systems. It sets a strong foundation for future advancements in big data analytics, paving the way for more efficient, responsive, and insightful data-driven decision-making.

Abstract

This project involves the implementation of a big data architecture designed for native analytics on fast and changing data. By integrating technologies like Apache Kafka, Apache Hadoop, Apache Spark, Apache Kudu, and Apache Impala, the architecture ensures efficient real-time data ingestion, processing, storage, and querying. The use case of real-time stock market data simulation demonstrates the system's capability to handle high-velocity data streams and provide low-latency responses, making it suitable for various real-world applications.

Keywords: Real-time analytics, Apache Kudu, Apache Impala, Datasets

Ce projet consiste en la mise en œuvre d'une architecture de big data conçue pour l'analyse native de données rapides et changeantes. En intégrant des technologies telles qu'Apache Kafka, Apache Hadoop, Apache Spark, Apache Kudu et Apache Impala, l'architecture garantit une ingestion, un traitement, un stockage et une interrogation des données en temps réel efficaces. Le cas d'utilisation de la simulation en temps réel des données boursières démontre la capacité du système à gérer des flux de données à grande vitesse et à fournir des réponses à faible latence, le rendant adapté à diverses applications réelles.

Mots clés: Analyse en temps réel, Apache Kudu, Apache Impala, Ensembles de données

يتضمن هذا المشروع تنفيذ بنية بيانات ضخمة مصممة للتحليلات الأصلية على البيانات السريعة والمتغيرة. من خلال دمج تقنيات مثل Apache Kafka ، Apache Hadoop ، Apache Spark ، Apache Kudu و Apache Impala ، تضمن البنية كفاءة في إدخال البيانات ومعالجتها وتخزينها واستجوابها في الوقت الفعلي. يوضح حالة الاستخدام لمحاكاة بيانات سوق الأسهم في الوقت الفعلي قدرة النظام على التعامل مع تدفقات البيانات عالية السرعة وتقديم استجابات منخفضة التأخير، مما يجعله مناسباً لمختلف التطبيقات في العالم الحقيقي.

الكلمات المفتاحية : تحليلات الوقت الفعلي، مجموعات البيانات ، Apache Impala ، Apache Kudu

Web Bibliography

- [1] Real-time analytics: Article <https://axual.com/technology-trends-of-the-future-real-time-data-streaming-is-the-way-to-go/>
- [2] Big Data Statistics: Article <https://www.linkedin.com/pulse/big-data-statistics-key-insights-future-trends-2024-beyond-nbrpe/>
- [3] Xiaomi Architecture: Article <https://bigdataschool.ru/blog/kudu-in-xiaomi-use-cases.html>
- [4] Kafka Documentation: Site <https://kafka.apache.org/documentation/>
- [5] NiFi Documentation: Site <https://nifi.apache.org/documentation/v2/>
- [6] Hadoop Documentation: Site <https://hadoop.apache.org/docs/current/>
- [7] Hadoop HDFS Erasure Coding: Site <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HDFSErasureCoding.html>
- [8] Erasure Coding Image : Site <https://data-flair.training/blogs/hadoop-hdfs-erasure-coding/>
- [9] Spark Documentation: Site <https://spark.apache.org/docs/latest/>
- [10] Kudu Documentation: Site <https://kudu.apache.org/docs/>
- [11] HBase Book: Site <https://hbase.apache.org/book.html>
- [12] Spark Streaming Programming Guide: Site <https://spark.apache.org/docs/latest/streaming-programming-guide>
- [13] Storm Tutorial: Site <https://storm.apache.org/releases/current/Tutorial.html>
- [14] Impala Overview: Site https://impala.apache.org/docs/build/asf-site-html/topics/impala_intro.html
- [15] Drill Documentation: Site <https://drill.apache.org/docs/>
- [16] Apache Kafka Architecture: Site <https://mail-narayank.medium.com/kafka-architecture-internal-d0b3334d1df>
- [17] Apache Hadoop Architecture: Site <https://subscription.packtpub.com/book/data/9781788999830/1/ch01v11sec03/what-hadoop-is-and-why-it-is-important>
- [18] HDFS3 Sink Connector Overview: Site <https://docs.confluent.io/kafka-connectors/hdfs3-sink/current/overview.html>
- [19] Apache Spark Architecture: Site <https://www.javatpoint.com/apache-spark-architecture>
- [20] Apache Impala Architecture: Site <https://impala.apache.org/overview.html>
- [21] Kudu-Impala Integration: Article <https://www.dataintensive.info/how-to-deploy-apache-kudu-impala-from-sources/>
- [22] Hue Browsing Documentation: Site <https://docs.gethue.com/user/browsing/>